

目录

1 引言.....	1
1.1 项目背景与设计任务.....	1
1.2 设计目标与支持指令集.....	1
1.3 设计环境与代码组成.....	2
2 CPU 总体架构设计.....	3
2.1 多周期 CPU 设计思想.....	3
2.2 CPU 顶层结构框图说明.....	4
2.3 数据通路总体设计.....	6
2.4 控制通路总体设计.....	7
2.5 顶层模块接口说明.....	8
2.6 顶层内部信号说明.....	10
3 指令字段解析与立即数生成设计.....	12
3.1 指令字段解析逻辑.....	12
3.2 Offset 字段生成逻辑.....	12
3.3 立即数扩展设计.....	12
3.4 指令字段对控制单元的影响.....	13
4 子模块详细设计.....	13
4.1 取指阶段模块设计.....	13
4.1.1 PC 模块.....	13
4.1.1.1 模块功能描述.....	13
4.1.1.2 接口信号说明.....	14
4.1.1.3 PCWrite 时序说明.....	14
4.1.1.4 PCWrite 设计思路详解.....	15
4.1.2 NPC 模块.....	15

Author:张琦伟

date:2026.5

Version:v1.0

4.1.2.1 模块功能描述.....	15
4.1.2.2 接口信号说明.....	16
4.1.2.3 NPCOp 时序说明.....	16
4.1.2.4 NPCOp 设计思路详解.....	17
4.1.3 IM 指令存储器模块.....	17
4.1.3.1 模块功能描述.....	17
4.1.3.2 接口信号说明.....	17
4.1.3.3 InsMemRW 时序说明.....	17
4.1.3.4 InsMemRW 设计思路详解.....	18
4.1.4 IR 指令寄存器模块.....	19
4.1.4.1 模块功能描述.....	19
4.1.4.2 接口信号说明.....	19
4.1.4.3 IRWrite 时序说明.....	19
4.1.4.4 IRWrite 设计思路详解.....	20
4.2 译码与寄存器读取相关模块设计.....	20
4.2.1 RF 寄存器堆模块.....	20
4.2.1.1 模块功能描述.....	20
4.2.1.2 接口信号说明.....	20
4.2.1.3 RFWrite 时序说明.....	21
4.2.1.4 RFWrite 设计思路详解.....	21
4.2.2 EXT 立即数扩展模块.....	21
4.2.2.1 模块功能描述.....	21
4.2.2.2 接口信号说明.....	22
4.2.2.3 ExtSel 时序说明.....	22
4.2.2.4 ExtSel 设计思路详解.....	22
4.3 执行通路模块设计.....	23
4.3.1 操作数寄存器 U_A、U_B.....	23
4.3.1.1 模块功能描述.....	23
4.3.1.2 锁存时序说明.....	23

4.3.1.3 设计思路详解.....	23
4.3.2 ALU 输入 A 选择器 MUX_2to1_A.....	23
4.3.2.1 模块功能描述.....	23
4.3.2.2 ALUSrcA 时序说明.....	24
4.3.2.3 ALUSrcA 设计思路详解.....	24
4.3.3 ALU 输入 B 选择器 MUX_3to1_B.....	24
4.3.3.1 模块功能描述.....	24
4.3.3.2 ALUSrcB 时序说明.....	24
4.3.3.3 ALUSrcB 设计思路详解.....	25
4.3.4 ALU 算术逻辑单元.....	25
4.3.4.1 模块功能描述.....	25
4.3.4.2 ALUOp 时序说明.....	25
4.3.4.3 ALUOp 设计思路详解.....	26
4.3.5 U_ALUOut 结果寄存器.....	26
4.3.5.1 模块功能描述.....	26
4.3.5.2 锁存时序说明.....	26
4.3.5.3 设计思路详解.....	26
4.4 访存通路模块设计.....	27
4.4.1 DM 数据存储器模块.....	27
4.4.1.1 模块功能描述.....	27
4.4.1.2 接口信号说明.....	27
4.4.1.3 DMCtrl 时序说明.....	27
4.4.1.4 DMCtrl 设计思路详解.....	27
4.5 写回通路模块设计.....	28
4.5.1 写回地址选择器 MUX_3to1.....	28
4.5.1.1 模块功能描述.....	28
4.5.1.2 RegSel 时序说明.....	28
4.5.1.3 RegSel 设计思路详解.....	28
4.5.2 写回数据选择器 MUX_3to1_LMD.....	28

4.5.2.1 模块功能描述.....	28
4.5.2.2 WDSel 时序说明.....	28
4.5.2.3 WDSel 设计思路详解.....	29
5 ControlUnit 控制单元详细设计.....	29
5.1 ControlUnit 模块功能概述.....	29
5.2 ControlUnit 输入输出接口说明.....	29
5.3 有限状态机设计.....	30
5.3.1 状态定义.....	30
5.3.2 状态转移逻辑.....	30
5.3.3 控制信号默认值设计.....	31
5.4 各状态控制信号生成详解.....	31
5.4.1 IT 状态控制信号.....	31
5.4.2 IF 状态控制信号.....	31
5.4.3 DAE 状态控制信号.....	32
5.4.4 MEM 状态控制信号.....	34
5.4.5 WB 状态控制信号.....	35
5.5 各重点控制信号时序说明与设计思路.....	36
5.5.1 PCWrite.....	36
5.5.2 IRWrite.....	36
5.5.3 InsMemRW.....	37
5.5.4 RFWrite.....	37
5.5.5 DMCtrl.....	38
5.5.6 ExtSel.....	38
5.5.7 ALUOp.....	39
5.5.8 NPCOp.....	39
5.5.9 ALUSrcA.....	40
5.5.10 ALUSrcB.....	40
5.5.11 RegSel.....	41
5.5.12 WDSel.....	41

5.6 控制信号默认值设计.....	41
5.6.1 默认值初始化的作用.....	41
5.6.2 防止组合逻辑锁存器的设计原因.....	42
5.6.3 默认控制信号与各状态覆盖赋值关系.....	43
5.7 各状态控制信号生成详解.....	44
5.7.1 IT 状态控制信号.....	44
5.7.2 IF 状态控制信号.....	45
5.7.3 DAE 状态控制信号.....	45
5.7.4 MEM 状态控制信号.....	48
5.7.5 WB 状态控制信号.....	49
5.8 各指令控制流程.....	49
5.8.1 R-type 指令控制流程.....	49
5.8.2 I-type 指令控制流程.....	50
5.8.3 lw 指令控制流程.....	50
5.8.4 sw 指令控制流程.....	51
5.8.5 beq 指令控制流程.....	51
5.8.6 bne 指令控制流程.....	52
5.8.7 jal 指令控制流程.....	53
5.8.8 jalr 指令控制流程.....	53
5.8.9 各类指令控制流程对比表.....	54
控制信号总表.....	55
6. 数据通路与控制信号配合分析.....	55
6.1 取指路径数据流与控制信号配合.....	55
6.2 译码路径数据流与控制信号配合.....	56
6.3 执行路径数据流与控制信号配合.....	57
6.4 访存路径数据流与控制信号配合.....	58
6.5 写回路路径数据流与控制信号配合.....	59
6.6 典型指令执行过程分析.....	60
7 系统设计总结.....	61

7.1 设计完成情况.....	61
7.2 系统结构特点.....	61
7.3 控制信号设计特点.....	61
7.4 数据通路设计特点.....	61
7.5 总结	62

快速预览简介

一、作品概述

本作品完成了一个基于 RISC-V 基础指令集的多周期 CPU 模块设计，采用 Verilog HDL 进行描述，顶层模块为 `riscv`。系统整体由取指通路、译码与控制通路、执行通路、访存通路和写回通路组成，内部包含 PC、NPC、IM、IR、RF、EXT、ALU、DM、多路选择器以及 ControlUnit 等子模块。顶层对外仅保留 `clk` 和 `rst` 两个输入端口，指令存储器和数据存储器均在 CPU 内部完成例化，便于进行模块化仿真、功能验证和后续综合分析。

本设计采用多周期 CPU 架构，将一条指令的执行过程拆分到多个状态中完成。控制单元采用有限状态机进行统一调度，状态包括 IT、IF、DAE、MEM 和 WB。其中，IT 只在复位后进入一次，用于完成初始取指准备；IF 负责指令锁存和部分顺序 PC 更新；DAE 将传统意义上的译码和执行合并到同一状态，用于完成指令字段判断、控制信号生成、ALU 运算、访存地址计算、分支比较和跳转目标选择；MEM 只服务于 `lw/sw` 指令的数据存储器访问；WB 则主要用于 R-type、I-type 和 `lw` 指令的寄存器写回，同时也承担分支和跳转类指令后的取指准备功能。

本作品支持的主要指令包括 R-type 运算指令、I-type 立即数指令、`lw`、`sw`、`beq`、`bne`、`jal` 和 `jalr`。其中，R-type 指令覆盖 `add`、`sub`、`and`、`or`、`xor`、`sll`、`srl`、`sra` 等常见算术逻辑和移位操作；I-type 指令覆盖 `addi`、`ori` 等立即数运算；访存类指令通过 DM 完成数据读写；分支和跳转类指令通过 `zero`、`NPCOp`、`PCWrite` 和 `PCA4` 等信号共同完成控制流改变和返回地址保存。

二、关键技术路线

本设计的核心技术路线可以概括为“集中式控制 + 模块化数据通路 + 多周期状态调度”。数据通路负责完成数据的读取、选择、运算、访存和写回；控制通路则根据当前状态、指令字段以及 ALU 的 `zero` 标志产生控制信号，驱动各个模块在对应周期内完成指定操作。

在取指通路中，PC 输出当前指令地址，IM 根据 `PC[11:2]` 输出指令 `in_ins`，IR 在 `IRWrite` 有效时将指令锁存为 `out_ins`。随后，顶层组合逻辑从 `out_ins` 中解析出 `opcode`、`Funct3`、`Funct7`、`rs1`、`rs2`、`rd`、`Imm12`、`Offset20` 和 `Offset` 等字段，为控制单元、寄存器堆、立即数扩展模块和下一地址生成模块提供输入。

在执行通路中，寄存器堆 RF 根据 `rs1` 和 `rs2` 读出 `RD1` 和 `RD2`。当前版本中，ALU 的主输入直接来自 `RD1` 和 `RD2`，其中 `RD1` 通过 `MUX_2to1_A` 送入 `ALUA` 端，`RD2`、`Imm32` 和 `Offset` 通过 `MUX_3to1_B` 选择后送入 `ALUB` 端。ALU 根据 `ALUOp` 完成算术运算、逻辑运算、移位运算、地址计算或分支比较，输出 `ALU_result` 和 `zero`。其中，`ALU_result` 一方面经过 `U_ALUOut` 锁存为 `ALU_result_r`，供后续访存或写回使用；另一方面也可直接送入 NPC 的 `rs_target` 端口，用于 `jalr` 指令的目标地址生成。

在访存通路中，`lw/sw` 指令在 DAE 状态先由 ALU 计算有效地址，并将结果锁存为 `ALU_result_r`。进入 MEM 状态后，DM 使用 `ALU_result_r[11:2]` 作为访问地址。对于 `lw` 指令，DM

读出的 RD 直接赋值给 DR_out，再通过写回选择器写入寄存器堆；对于 sw 指令，写入数据来自 RD2_r，用于保证存储器写入时使用的是当前指令对应的 rs2 数据。

在写回通路中，MUX_3to1 根据 RegSel 选择写回地址 WR，MUX_3to1_LMD 根据 WDSel 在 ALU_result_r、DR_out 和 PCA4 之间选择写回数据 WD。当 RFWrite 有效时，寄存器堆在时钟沿完成写入。R-type 和 I-type 指令在 WB 状态写回 ALU_result_r；lw 指令在 WB 状态写回 DR_out；jal/jalr 则在 DAE 状态将 PCA4 写回目标寄存器。

三、设计亮点

本设计的第一项亮点是将译码与执行合并为 DAE 状态。传统多周期 CPU 往往将译码和执行拆分为独立阶段，而本设计把指令字段解析、控制信号生成、ALU 运算、访存地址计算、分支比较和跳转控制集中到 DAE 状态中完成。这样做可以减少普通指令在状态机中的停留阶段，使 R-type、I-type、Branch、jal 和 jalr 等指令都能够按照 IF → DAE → WB → IF 的路径完成主要执行流程；只有 lw/sw 这类确实需要访问数据存储器的指令才进入 MEM 状态。该设计避免了所有指令都统一经过最长状态路径，提高了状态调度的针对性，也让控制流程更加集中清晰。

本设计的第二项亮点是采用分类型 PC 更新策略。对于 R-type、I-type、lw 和 sw 等普通顺序指令，下一条指令地址固定为 PC+4，因此控制器可以在 IF 状态根据 in_ins[6:0] 提前置 PCWrite=1，使 PC 提前更新到下一条顺序指令地址。对于 beq/bne、jal 和 jalr 等控制转移类指令，目标地址需要等到 DAE 状态完成比较或计算后才能确定，因此这些指令在 IF 状态保持 PCWrite=0，在 DAE 状态再通过 NPCOp 和 PCWrite 完成 PC 更新。这样的设计既保证了普通顺序指令取指的连续性，又避免了分支和跳转类指令在目标地址尚未确定时错误更新 PC。

本设计的第三项亮点是控制信号时序划分明确。PCWrite、IRWrite、InsMemRW、RFWrite、DMCtrl、ExtSel、ALUOp、NPCOp、ALUSrcA、ALUSrcB、RegSel 和 WDSel 均由 ControlUnit 统一生成，并在不同状态下发挥作用。其中，IRWrite 只在 IF 状态有效，用于保证当前指令稳定锁存；InsMemRW 根据指令类型在 IT、DAE 或 WB 状态有效，用于配合初始取指、顺序预取和跳转后取指；RFWrite 对 R-type、I-type 和 lw 在 WB 状态有效，而对 jal/jalr 在 DAE 状态有效。通过这种“默认值 + 状态覆盖”的方式，控制器能够避免无效写入和误访问，使系统在不同指令路径下保持稳定。

本设计的第四项亮点是数据通路结构清晰，模块之间职责明确。取指通路负责地址和指令流转，译码通路负责字段解析和立即数生成，执行通路复用 ALU 完成运算、地址计算和分支比较，访存通路只服务于 lw/sw，写回通路统一管理寄存器写地址和写数据选择。各模块之间通过明确的端口和信号连接，既便于仿真观察，也便于在报告中对数据流、控制流以及子模块交互关系进行说明。

1 引言

1.1 项目背景与设计任务

本设计面向 RISC-V 基础指令集处理器的实现需求，完成了一个基于 Verilog HDL 的多周期 CPU 模块设计。RISC-V 是一种开源精简指令集架构，具有指令格式规整、模块化程度高、扩展性强等特点，适合用于 CPU 原理学习、数字系统设计以及集成电路相关课程实践。

本设计采用多周期 CPU 架构，将指令执行过程划分到多个状态中完成。ControlUnit 采用 IT、IF、DAE、MEM、WB 五个状态，其中 DAE 状态将传统意义上的译码和执行功能合并，用于完成指令字段判断、ALU 运算、地址计算、分支比较和跳转控制。

本设计的顶层模块为 riscv，系统对外只保留 clk 和 rst 两个输入端口，内部由 PC、NPC、IM、IR、RF、EXT、ALU、DM、MUX 和 ControlUnit 等模块构成。ControlUnit 采用有限状态机，根据当前状态和指令字段生成各类控制信号，驱动数据通路在不同阶段完成对应操作。

1.2 设计目标与支持指令集

本设计的目标是实现一个结构清晰、功能完整、控制逻辑明确、便于仿真和综合的 RISC-V 多周期 CPU。整体设计遵循模块化思想，将数据通路和控制通路分开设计，各子模块之间通过明确的端口和信号连接，使取指、译码、执行、访存和写回等功能能够在不同状态下有序完成。

本设计的主要目标如下：

一是完成多周期 CPU 的顶层结构设计，使 CPU 能够按照 IT、IF、DAE、MEM、WB 等状态完成指令执行。其中，IT 用于复位后的初始化取指准备，IF 用于指令锁存和部分顺序 PC 更新，DAE 用于完成译码、控制信号生成、ALU 运算、地址计算、分支判断和跳转控制，MEM 用于 lw/sw 指令的数据存储器访问，WB 用于运算结果或访存数据的写回；

二是完成指令字段解析，包括 opcode、Funct3、Funct7、rs1、rs2、rd、Imm12、Offset20 和 Offset 等信号的提取，为控制单元、寄存器堆、立即数扩展模块和跳转地址生成模块提供必要输入；

三是完成主要数据通路设计，包括取指通路、寄存器读写通路、立即数扩展通路、ALU 运算通路、数据存储器访问通路和写回通路。当前版本中，ALU 的主输入直接来自寄存器堆输

出的 RD1 和 RD2, U_A 和 U_B 保留锁存路径, 其中 RD2_r 主要用于 sw 指令的存储器写入数据;

四是完成集中式控制单元设计, 通过有限状态机产生 PCWrite、IRWrite、RFWrite、DMCtrl、InsMemRW、ExtSel、ALUOp、NPCOp、ALUSrcA、ALUSrcB、RegSel 和 WDSel 等控制信号, 同时引入 in_ins[6:0] 作为控制单元输入, 使其能够在 IF 阶段提前判断普通顺序指令是否需要更新 PC;

五是保证各重点控制信号在不同状态下的作用时机明确, 能够正确驱动各模块完成指令执行。对于普通顺序指令, PCWrite 可在 IF 阶段完成顺序更新; 对于分支和跳转类指令, 则在 DAE 阶段结合 zero、Funct3 和 NPCOp 完成 PC 更新控制, 从而保证不同类型指令都能按照预期路径执行。

当前设计主要支持以下 RISC-V 指令类型:

指令类型	代表指令	功能说明
R-type	add、sub、and、or、xor、sll、srl、sra	寄存器之间的算术、逻辑和移位运算
I-type	addi、ori	寄存器与立即数之间的算术或逻辑运算
Load	lw	从数据存储器读取数据并写回寄存器
Store	sw	将寄存器中的数据写入数据存储器
Branch	beq、bne	根据两个寄存器的比较结果决定是否跳转
Jump	jal、jalr	无条件跳转, 并将 PC+4 作为返回地址写回

表 1-1 指令说明

1.3 设计环境与代码组成

本设计采用 Verilog HDL 编写, 主要代码由顶层模块和控制单元模块组成。顶层模块 riscv 负责完成各个功能子模块的例化和信号连接, 控制单元 ControlUnit 负责实现多周期有限状态机和控制信号生成。

代码组成如下:

文件或模块	作用
riscv.v	CPU 顶层模块，完成各子模块实例化和信号互连
ControlUnit	控制单元模块，采用 FSM 生成各类控制信号
PC	程序计数器，保存当前指令地址
NPC	下一 PC 生成模块，计算 PC+4、分支目标和跳转目标
IM	指令存储器，根据 PC 地址输出指令
IR	指令寄存器，锁存当前指令
RF	寄存器堆，提供双读单写寄存器访问
EXT	立即数扩展模块，生成 32 位立即数
Flopr	通用寄存器模块，用于锁存操作数和 ALU 结果
ALU	算术逻辑单元，完成运算、比较和地址计算
DM	数据存储器，完成 lw/sw 访存
MUX 系列	完成 ALU 输入选择、写回地址选择和写回数据选择

表 1-2 代码模块作用

2 CPU 总体架构设计

2.1 多周期 CPU 设计思想

多周期 CPU 的核心思想是将一条指令的执行过程拆分到多个时钟周期中完成，每个周期只完成当前阶段所需要的一部分操作。这样设计的好处是可以复用 PC、NPC、寄存器堆、ALU、数据存储器以及写回选择器等硬件资源，不需要像单周期 CPU 那样按照最长指令的完整延迟来确定时钟周期，因此整体结构更加清晰，也更便于后续对控制信号进行分析和调试。

在本设计中，一条指令最多经过以下五个状态：

状态	名称	主要功能
IT	初始化取指准备状态	系统复位后首先进入该状态，用于启动初始取指过程，为后续 IF 状态锁存指令做准备
IF	取指与指令锁存状态	根据当前 PC 从 IM 中读取指令，并在 IRWrite 有效时将 in_ins 写入 IR；对于 R-type、I-type、lw、sw 等普通顺序指令，可在该阶段提前更新 PC
DAE	译码与执行合并状态	解析 opcode、Funct3、Funct7 等指令字段，产生对应控制信号；同时完成 ALU 运算、访存地址计算、分支比较以及 jal/jalr 跳转目标选择
MEM	访存状态	只对 lw 和 sw 指令有效，lw 从数据存储器 DM 读取数据，sw 将寄存器数据写入数据存储器 DM
WB	写回或取指准备状态	对 R-type、I-type 和 lw 指令完成寄存器写回；对于分支和跳转类指令，该状态主要用于配合后续取指流程，使状态机回到 IF

表 2-1 状态功能

2.2 CPU 顶层结构框图说明

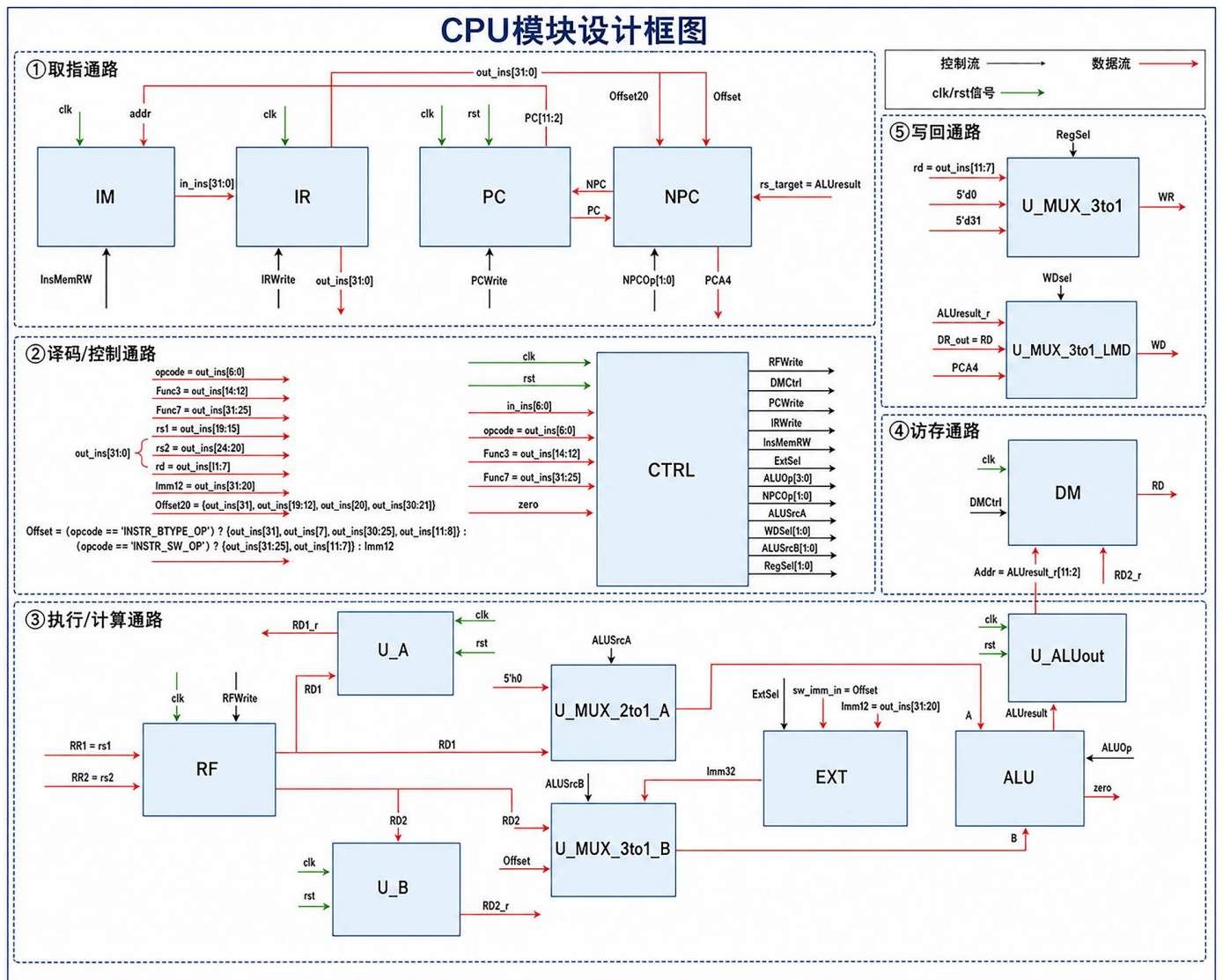


图 1-设计框图

CPU 顶层结构可以按照数据流和控制流划分为五个主要部分：

一是取指通路，由 PC、NPC、IM 和 IR 构成，主要负责产生当前指令地址、从指令存储器中读取指令，并通过 IR 保存当前正在执行的指令。其中，PC 保存当前指令地址，NPC 根据 NPCOp 产生下一条指令地址，IM 根据 PC[11:2] 输出指令 in_ins，IR 则在 IRWrite 有效时将指令锁存为 out_ins，供后续字段解析和控制单元使用。

二是译码与控制通路，由指令字段解析逻辑、ControlUnit 和 EXT 立即数扩展模块构成，主要负责识别指令类型、生成立即数，并产生整个 CPU 运行所需的控制信号。指令字段解析逻辑从 out_ins 中提取 opcode、Func3、Func7、rs1、rs2、rd、Imm12、Offset20 和 Offset 等信号，其中 opcode、Func3、Func7 送入 ControlUnit 用于控制信号生成，Imm12 和 Offset

送入 EXT 完成立即数扩展。同时,当前版本的 ControlUnit 还接收 in_ins[6:0],用于在 IF 阶段提前判断普通顺序指令是否可以更新 PC。

三是执行通路,由 RF、U_A、U_B、MUX_2to1_A、MUX_3to1_B、ALU 和 U_ALUOut 构成,主要完成寄存器读出、ALU 操作数选择、算术逻辑运算、地址计算和分支比较。当前顶层代码中,ALU 的主输入直接来自寄存器堆输出的 RD1 和 RD2,其中 RD1 送入 MUX_2to1_A, RD2 送入 MUX_3to1_B; U_A 和 U_B 仍然保留用于锁存寄存器读出数据, RD1_r 主要作为保留路径说明, RD2_r 则用于 sw 指令在访存阶段向数据存储器写入数据。

四是访存通路,由 DM 数据存储器构成,主要服务于 lw 和 sw 指令。lw/sw 指令在 DAE 阶段通过 ALU 计算访存地址,地址结果经 U_ALUOut 锁存为 ALU_result_r,随后在 MEM 阶段使用 ALU_result_r[11:2] 访问数据存储器。对于 sw 指令,写入数据来自 RD2_r; 对于 lw 指令,数据存储器输出 RD,并通过 assign DR_out = RD 直接送入写回数据选择器。

五是写回通路,由写回地址选择器 MUX_3to1 和写回数据选择器 MUX_3to1_LMD 构成,主要负责选择寄存器堆的写地址和写回数据。MUX_3to1 根据 RegSel 在 rd、x0 和 x31 之间选择写回地址 WR; MUX_3to1_LMD 根据 WDSel 在 ALU_result_r、DR_out 和 PCA4 之间选择写回数据 WD。当 RFWrite 有效时,RF 在时钟沿将 WD 写入 WR 指定的寄存器,从而完成 R-type、I-type、lw 以及跳转链接类指令的写回操作。

2.3 数据通路总体设计

本设计的数据通路以 32 位数据宽度为主,顶层模块通过各功能子模块之间的连线完成取指、译码、执行、访存和写回等操作。指令首先由 PC 给出访问地址,IM 根据 PC[11:2] 输出指令 in_ins,随后 IR 在 IRWrite 有效时将该指令锁存为 out_ins。out_ins 再通过顶层组合逻辑被解析为 opcode、Funct3、Funct7、rs1、rs2、rd、Imm12、Offset20 和 Offset 等字段,为控制单元和数据通路提供后续执行所需的信息。

寄存器堆 RF 根据 rs1 和 rs2 读出两个源操作数 RD1 和 RD2。RD1 和 RD2 直接作为 ALU 输入选择器的主要数据来源,其中 RD1 送入 MUX_2to1_A, RD2 送 MUX_3to1_B。MUX_2to1_A 根据 ALUSrcA 在 RD1 和常数 0 之间选择 ALU 的 A 端输入 MUX_3to1_B 根据 ALUSrcB 在 RD2、Imm32 和 Offset 之间选择 ALU 的 B 端输入。随后,ALU 根据 ALUOp 完成算术运算、逻辑运算、移位运算、访存地址计算或分支比较,并输出 ALU_result 和 zero 信号。

U_A 和 U_B 用于对 RD1 和 RD2 进行锁存，输出分别为 RD1_r 和 RD2_r。ALU 直接使用 RD1 和 RD2。其中，RD1_r 目前主要作为保留锁存路径存在，RD2_r 则在 sw 指令的访存阶段作为数据存储器 DM 的写入数据使用，保证写存储器时使用的是当前指令对应的 rs2 数据。ALU 的结果经过 U_ALUOut 锁存为 ALU_result_r，该结果既可以作为 lw/sw 指令访问数据存储器的地址，也可以作为 R-type 和 I-type 指令写回寄存器堆的数据来源。

主要数据流如下：

PC → IM → IR → 指令字段解析

out_ins → rs1 / rs2 / rd / Imm12 / Offset20 / Offset

rs1 / rs2 → RF → RD1 / RD2

RD1 → MUX_2to1_A → A

RD2 / Imm32 / Offset → MUX_3to1_B → B

A / B → ALU → ALU_result → U_ALUOut → ALU_result_r

ALU_result_r[11:2] → DM

RD2_r → DM.WD

DM.RD → DR_out

ALU_result_r / DR_out / PCA4 → MUX_3to1_LMD → WD → RF

rd / x0 / x31 → MUX_3to1 → WR → RF

2.4 控制通路总体设计

控制通路的核心是 ControlUnit。该模块输入当前指令字段和 ALU 的 zero 标志，并根据有限状态机当前状态输出控制信号。

ControlUnit 的输入信号包括：

输入信号	来源	作用
clk	顶层输入	控制状态机同步更新
rst	顶层输入	系统复位

输入信号	来源	作用
zero	ALU	分支判断标志
opcode	指令字段解析	判断指令类型
Funct3	指令字段解析	判断具体运算或分支类型
Funct7	指令字段解析	区分 add/sub、srl/sra 等操作
in_ins[6:0]	IM 输出指令低 7 位	在 IF 状态提前判断当前取出的指令类型，决定普通顺序指令是否提前更新 PC

表 2-2 ControlUnit 的输入信号

ControlUnit 的输出信号包括：

控制信号	作用
PCWrite	控制 PC 是否写入 NPC
IRWrite	控制 IR 是否锁存新指令
InsMemRW	控制指令存储器访问
RFWrite	控制寄存器堆写回
DMCtrl	控制数据存储器读写
ExtSel	控制立即数扩展方式
ALUOp	控制 ALU 运算类型
NPCOp	控制下一 PC 的来源
ALUSrcA	控制 ALU A 端输入来源
ALUSrcB	控制 ALU B 端输入来源
RegSel	控制写回寄存器地址来源
WDSel	控制写回数据来源

表 2-3 ControlUnit 的输出信号

2.5 顶层模块接口说明

顶层模块 riscv 的外部接口如下：

端口名	方向	位宽	功能
clk	input	1	系统时钟
rst	input	1	系统复位，高有效

表 2-4 外部接口

本设计的指令存储器和数据存储器均在 CPU 内部实例化，因此顶层没有暴露外部存储器总线接口。

2.6 顶层内部信号说明

信号名	位宽	来源	目的	功能
PC	32	PC 模块	IM、NPC	当前指令地址，送入指令存储器取指，同时送入 NPC 参与下一条指令地址计算
NPC	32	NPC 模块	PC	下一条指令地址，在 PCWrite 有效时写入 PC
PCA4	32	NPC 模块	MUX_3to1_LMD	PC + 4, 主要用于 jal/jalr 指令的返回地址写回
in_ins	32	IM	IR、ControlUnit	指令存储器输出指令，其中完整 in_ins 送入 IR，低 7 位 in_ins[6:0] 送入 ControlUnit, 用于 IF 阶段提前判断指令类型
out_ins	32	IR	指令字段解析逻辑	当前锁存的执行指令，用于解析 opcode、Funct3、Funct7、rs1、rs2、rd 和立即数字段
opcode	7	out_ins[6:0]	ControlUnit	指令操作码，用于判断当前执行指令类型
Funct3	3	out_ins[14:12]	ControlUnit	功能字段，用于区分具体 ALU 运算、分支类型等
Funct7	7	out_ins[31:25]	ControlUnit	功能字段，用于区分 add/sub、srl/sra 等运算
rs1	5	out_ins[19:15]	RF	源寄存器 1 地址
rs2	5	out_ins[24:20]	RF	源寄存器 2 地址
rd	5	out_ins[11:7]	MUX_3to1	目的寄存器地址，作为普通写回地址来源
Imm12	12	out_ins[31:20]	EXT、Offset 选择逻辑	I-type 立即数字段，也可作为默认偏移字段来源
Offset20	20	字段拼接逻辑	NPC	J-type 跳转偏移字段，用于 jal 目标地址计算
Offset	12	字段拼接与选择逻辑	EXT、MUX_3to1_B、NPC	B-type、S-type 或 I-type 偏移字段，根据 opcode 选择生成
RD1	32	RF	MUX_2to1_A、U_A、NPC	源操作数 1；当前版本中直接作为 ALU A 端主输入，同时也送入 U_A 锁存，并作为 NPC 的 rs 基址来源
RD2	32	RF	MUX_3to1_B、U_B	源操作数 2；当前版本中直接作为 ALU B 端主输入，同时送入 U_B 锁存

信号名	位宽	来源	目的	功能
RD1_r	32	U_A	保留路径	RD1 的锁存结果,当前顶层中未作为 ALU 主输入使用,主要作为辅助锁存或保留路径
RD2_r	32	U_B	DM	RD2 的锁存结果,主要用于 sw 指令在 MEM 阶段向数据存储器写入数据
Imm32	32	EXT	MUX_3to1_B	扩展后的 32 位立即数,用于 I-type 运算和 lw/sw 地址计算
A	32	MUX_2to1_A	ALU	ALU A 端输入,由 ALUSrcA 控制选择,当前主要来源为 RD1
B	32	MUX_3to1_B	ALU	ALU B 端输入,由 ALUSrcB 控制选择,可来自 RD2、Imm32 或 Offset
ALU_result	32	ALU	U_ALUOut、NPC	ALU 当前组合输出结果,一方面送入 U_ALUOut 锁存,另一方面直接送入 NPC 的 rs_target,用于 jalr 等目标地址选择
zero	1	ALU	ControlUnit	ALU 比较结果标志,主要用于 beq/bne 分支判断
ALU_result_r	32	U_ALUOut	DM、MUX_3to1_LMD	锁存后的 ALU 结果,用于 lw/sw 的访存地址,也可作为 R-type、I-type 指令的写回数据
RD	32	DM	DR_out	数据存储器读出数据,主要用于 lw 指令
DR_out	32	RD 直接赋值	MUX_3to1_LMD	当前等同于 RD,即 assign DR_out = RD,没有单独使用 U_DR 寄存器
WR	5	MUX_3to1	RF	寄存器堆写地址,由 RegSel 控制选择,可来自 rd、x0 或 x31
WD	32	MUX_3to1_LMD	RF	寄存器堆写数据,由 WDSel 控制选择,可来自 ALU_result_r、DR_out 或 PCA4

表 2-5 内部信号说明

3 指令字段解析与立即数生成设计

3.1 指令字段解析逻辑

本设计中，指令由 IR 模块输出为 `out_ins`，随后在顶层通过组合逻辑解析各个字段。字段划分如下：

字段	代码表达式	功能
opcode	<code>out_ins[6:0]</code>	判断指令类型
rd	<code>out_ins[11:7]</code>	目的寄存器地址
Funct3	<code>out_ins[14:12]</code>	功能字段
rs1	<code>out_ins[19:15]</code>	源寄存器 1 地址
rs2	<code>out_ins[24:20]</code>	源寄存器 2 地址
Funct7	<code>out_ins[31:25]</code>	功能字段
Imm12	<code>out_ins[31:20]</code>	I-type 立即数
Offset20	<code>{out_ins[31],out_ins[19:12],out_ins[20],out_ins[30:21]}</code>	J-type 跳转偏移字段

表 3-1ControlUnit 的输入信号

指令字段解析是连接数据通路和控制通路的重要部分。`opcode`、`Funct3` 和 `Funct7` 送入 `ControlUnit`，用于决定当前指令类型和 ALU 运算方式；`rs1`、`rs2` 和 `rd` 则用于寄存器堆读写。

3.2 Offset 字段生成逻辑

当前顶层代码中，`Offset` 根据信号 `opcode` 进行选择：

如果 `opcode` 为 B-type: `Offset = {out_ins[31], out_ins[7], out_ins[30:25], out_ins[11:8]}`

如果 `opcode` 为 sw: `Offset = {out_ins[31:25], out_ins[11:7]}`

否则: `Offset = Imm12`

该设计将 B-type、S-type 和 I-type 中不同位置的立即数字段统一整理为 `Offset`，便于后续送入 EXT、NPC 或 ALU 输入选择器。由于 RISC-V 不同类型指令的立即数字段并不连续，因此需要在顶层对字段重新拼接。

3.3 立即数扩展设计

立即数扩展由 EXT 模块完成。EXT 的输入包括 `Imm12`、`Offset` 和 `ExtSel`，输出为 32 位的

Imm32。

对于 addi、lw、sw 等指令，立即数通常作为有符号数参与加法或地址计算，因此需要符号扩展；对于 ori 指令，立即数作为逻辑操作数参与按位或运算，因此采用零扩展。

指令类型	立即数来源	扩展方式	用途
addi	Imm12	符号扩展	ALU 加法
ori	Imm12	零扩展	ALU 逻辑或
lw	Imm12/Offset	符号扩展	地址偏移
sw	S-type Offset	符号扩展	地址偏移
beq/bne	B-type Offset	偏移量拼接	分支目标
jal	Offset20	J-type 偏移	跳转目标

表 3-2 立即数拓展

3.4 指令字段对控制单元的影响

ControlUnit 主要根据 opcode 判断指令大类，根据 Funct3 和 Funct7 进一步确定具体操作。例如，R-type 指令进入 DAE 阶段后，如果 Funct3=000，则通过 Funct7[5] 区分 add 和 sub；如果 Funct3=101，则通过 Funct7[5] 区分 srl 和 sra。

Branch 指令则需要结合 Funct3 和 zero 信号判断是否跳转。beq 在 zero=1 时跳转，bne 在 zero=0 时跳转。由此可见，指令字段解析不仅影响数据通路的地址和数据选择，也直接影响控制通路的状态转移和控制信号输出。

4 子模块详细设计

4.1 取指阶段模块设计

4.1.1 PC 模块

4.1.1.1 模块功能描述

PC 模块是程序计数器，用于保存当前正在执行的指令地址。PC 的输出连接到 IM 指令存储器和 NPC 下一地址生成模块。IM 根据 PC 读取当前指令，NPC 根据 PC 和控制信号计算下一条指令地址。

PC 模块在复位时清零，在正常运行时由 PCWrite 控制是否更新。当 PCWrite=1 时，PC 在时钟上升沿加载 NPC；当 PCWrite=0 时，PC 保持原值不变。

4.1.1.2 接口信号说明

信号名	方向	位宽	描述
clk	input	1	系统时钟
rst	input	1	高有效复位
PCWrite	input	1	PC 写使能
NPC	input	32	下一条指令地址
PC	output	32	当前指令地址

表 4-1 接口信号

4.1.1.3 PCWrite 时序说明

PCWrite 是 PC 模块的重点控制信号，用于决定 PC 是否在当前时钟沿更新。

在 IF 状态下，ControlUnit 会根据 in_ins[6:0] 判断当前从指令存储器读出的指令类型。对于 R-type、I-type、lw 和 sw 这类顺序执行指令，控制器会在 IF 阶段置 PCWrite=1，使 PC 提前更新为下一条顺序指令地址。对于 beq/bne、jal/jalr 等可能改变程序执行流的指令，IF 阶段不更新 PC，而是在 DAE 阶段完成分支判断或跳转目标选择后，再置 PCWrite=1 更新 PC。

指令类型	PCWrite 有效阶段	说明
R-type	IF	普通顺序执行指令，在 IF 阶段根据 in_ins[6:0] 判断后提前更新 PC，下一地址通常为 PC+4
I-type	IF	普通顺序执行指令，在 IF 阶段提前更新 PC
lw	IF	IF 阶段提前更新 PC，后续 DAE、MEM、WB 阶段继续完成地址计算、访存和写回
sw	IF	IF 阶段提前更新 PC，后续 DAE、MEM 阶段完成地址计算和数据写入
beq/bne	DAE	DAE 阶段完成寄存器比较，根据 zero 和 Funct3 判断是否选择分支目标地址，然后更新 PC
jal	DAE	DAE 阶段选择 J-type 跳转目标地址，并更新 PC
jalr	DAE	DAE 阶段由 ALU 计算 rs1 + imm，并通过 NPC 选择寄存器间接跳转目标，然后更新 PC

表 4-2 PCWrite 时序说明

4.1.1.4 PCWrite 设计思路详解

PCWrite 设计思路可以概括为：顺序指令提前更新 PC，控制转移指令等待目标确定后再更新 PC。

对于 R-type、I-type、lw 和 sw 这类普通顺序执行指令，下一条指令地址一定是 PC+4，不需要等待 ALU 比较结果或跳转目标计算。因此，控制器可以在 IF 阶段根据 in_ins[6:0] 判断指令类型后，直接令 PCWrite=1，使 PC 提前更新到下一条顺序指令地址。此时当前指令已经通过 IRWrite 被锁存到 IR 中，后续 DAE、MEM 和 WB 阶段使用的是稳定的 out_ins，所以 PC 的提前更新不会破坏当前指令的执行。

对于 beq/bne 分支指令，是否跳转需要等到 DAE 阶段由 ALU 完成两个寄存器的比较后才能确定。如果在 IF 阶段就直接更新为 PC+4，就可能在分支成立时产生错误的顺序取指。因此，分支指令在 IF 阶段保持 PCWrite=0，到 DAE 阶段根据 zero 和 Funct3 判断分支是否成立，再通过 NPCOp 选择 PC+4 或分支目标地址，并置 PCWrite=1 更新 PC。

对于 jal 和 jalr 跳转指令，下一条指令地址也不是简单的 PC+4。jal 需要根据 Offset20 生成 J-type 跳转目标地址，jalr 需要通过 ALU 计算 rs1 + imm 得到寄存器间接跳转目标。因此，这两类指令同样在 IF 阶段不更新 PC，而是在 DAE 阶段设置对应的 NPCOp，并置 PCWrite=1 完成跳转地址写入。

这种设计使普通顺序指令可以提前准备下一条指令地址，提高取指流程的连续性；同时又避免分支和跳转指令在目标地址尚未确定时错误更新 PC，从而保证不同类型指令都能按照正确的控制路径执行。

4.1.2 NPC 模块

4.1.2.1 模块功能描述

NPC 模块用于产生下一条指令地址。它根据当前 PC、分支偏移量、跳转偏移量、寄存器基地址以及控制信号 NPCOp 选择下一 PC 的来源。NPC 模块还输出 PCA4，即 PC+4。该信号除了用于顺序执行之外，还用于 jal 和 jalr 指令的链接地址写回。

4.1.2.2 接口信号说明

信号名	方向	位宽	描述
PC	input	32	当前指令地址
NPCOp	input	2	下一地址选择控制
Offset12	input	32	分支或短偏移
Offset20	input	20	J-type 跳转偏移
rs	input	32	jalr 使用的寄存器基地址
rs_target	input	32	ALU 结果相关跳转目标
PCA4	output	32	PC+4
NPC	output	32	下一条指令地址

表 4-3 接口信号说明

4.1.2.3 NPCOp 时序说明

NPCOp 是 NPC 模块的核心选择信号。由于 NPC 模块本身主要是组合逻辑，NPCOp 选择出的结果并不会立即改变 PC，只有当 PCWrite=1 时，当前 NPC 输出才会在时钟沿写入 PC。普通顺序指令在 IF 阶段提前更新 PC，此时 NPCOp 保持默认的顺序地址选择；分支和跳转类指令在 IF 阶段不更新 PC，而是在 DAE 阶段完成判断或目标地址选择后，再配合 PCWrite 更新 PC。

指令类型	NPCOp 设置阶段	PCWrite 有效阶段	作用
R-type	IF	IF	NPCOp 保持默认顺序地址选择，PC 更新为 PC+4
I-type	IF	IF	NPCOp 保持默认顺序地址选择，PC 更新为 PC+4
lw	IF	IF	IF 阶段提前更新 PC，后续继续完成地址计算、访存和写回
sw	IF	IF	IF 阶段提前更新 PC，后续完成地址计算和数据写入
beq/bne	DAE	DAE	在 DAE 阶段根据 zero 和 Funct3 判断是否选择分支目标地址
jal	DAE	DAE	在 DAE 阶段选择 J-type 跳转目标地址
jalr	DAE	DAE	在 DAE 阶段选择由 ALU_result 产生的寄存器间接跳转目标

表 4-4 时序说明

4.1.2.4 NPCOp 设计思路详解

NPC 模块将顺序执行、条件分支和无条件跳转统一到一个地址生成模块中。ControlUnit 不直接计算下一地址，而是通过 NPCOp 告诉 NPC 选择哪一种地址来源。这样可以降低控制单元复杂度，也便于后续扩展更多跳转方式。

对于普通指令，下一条地址为 PC+4；对于 beq/bne，DAE 阶段由 ALU 产生 zero，ControlUnit 根据 zero 判断是否选择分支目标；对于 jal，下一地址来自 J-type 偏移计算；对于 jalr，下一地址来自寄存器基地址相关计算。

4.1.3 IM 指令存储器模块

4.1.3.1 模块功能描述

IM 模块是指令存储器，用于根据 PC 地址读取当前指令。顶层代码中，IM 的地址输入为 PC[11:2]，输出为 in_ins，随后送入 IR 模块。由于 RISC-V 指令长度为 32 位，即 4 字节，因此 PC 的低两位用于字节对齐，实际访问指令存储器时使用 PC[11:2] 作为字地址。

4.1.3.2 接口信号说明

信号名	方向	位宽	描述
clk	input	1	时钟信号
addr	input	10	指令地址，来自 PC[11:2]
InsMemRW	input	1	指令存储器访问控制
Ins	output	32	输出指令

表 4-5 接口信号说明

4.1.3.3 InsMemRW 时序说明

InsMemRW 根据状态机执行过程承担指令读取和下一条指令预取的作用。复位后，状态机首先进入 IT 状态，此时 InsMemRW=1，用于启动初始指令读取；随后进入 IF 状态，IF 阶段主要通过 IRWrite 将当前 in_ins 锁存到 IR 中。

对于 R-type、I-type、lw 和 sw 等普通顺序执行指令，ControlUnit 会在 DAE 阶段将 InsMemRW 置为 1，用于配合已经提前更新的 PC 读取下一条顺序指令。对于 beq/bne、jal 和 jalr 等分支或跳转类指令，由于它们需要在 DAE 阶段确定新的 PC，因此会在后续 WB 阶

段将 InsMemRW 置为 1，用于从更新后的目标地址处读取下一条指令。

状态/指令情况	InsMemRW	说明
IT	1	复位后的初始化取指准备状态，用于启动第一次指令读取
IF	0	主要通过 IRWrite=1 将当前 in_ins 锁存到 IR，不在该状态单独启动新的指令读取
DAE，且当前指令为 R-type/I-type/lw/sw	1	普通顺序指令在该阶段提前读取下一条顺序指令
DAE，且当前指令为 beq/bne/jal/jalr	0	控制转移类指令需要先确定目标 PC，不在该阶段直接读取新指令
MEM	0	访存阶段主要用于 lw/sw 访问数据存储器，不访问指令存储器
WB，且当前指令为 beq/bne/jal/jalr	1	分支或跳转目标地址已经确定，用于读取目标地址处的下一条指令
WB，且当前指令为 R-type/I-type/lw	0	普通顺序指令的下一条指令已在前面阶段进行读取准备，此处不再启动指令读取

表 4-6 时序说明

4.1.3.4 InsMemRW 设计思路详解

InsMemRW 用于控制指令存储器 IM 的访问过程。虽然本设计中的 IM 主要作为只读指令存储器使用，但保留 InsMemRW 可以让取指行为由控制单元统一管理，使指令读取、指令锁存和 PC 更新之间的配合关系更加清楚。

当前设计思路采用了类似预取的方式。对于普通顺序执行指令，PC 会在 IF 阶段提前更新为 PC+4，而当前指令已经通过 IRWrite 锁存在 IR 中，因此后续执行阶段不会受到 PC 变化影响。进入 DAE 阶段后，控制器将 InsMemRW 置为 1，使 IM 可以根据已经更新后的 PC 读取下一条顺序指令，为下一次 IF 阶段锁存指令做准备。

对于 beq/bne、jal 和 jalr 等控制转移类指令，PC 不能在 IF 阶段简单更新为 PC+4，而是必须等到 DAE 阶段完成分支判断或跳转目标计算后才能确定。因此，这类指令在 DAE 阶段不会立即置 InsMemRW=1，而是在 PC 更新完成后，通过后续 WB 阶段将 InsMemRW 置为 1，从新的目标地址处读取下一条指令。

这种设计使普通顺序指令能够提前准备下一条指令，提高取指流程的连续性；同时又保证分支和跳转指令不会在目标地址尚未确定时错误访问指令存储器。

4.1.4 IR 指令寄存器模块

4.1.4.1 模块功能描述

IR 模块用于锁存 IM 输出的指令。由于多周期 CPU 中一条指令需要多个周期才能执行完成，如果没有 IR 保存当前指令，那么后续阶段可能无法继续使用正确的指令字段。因此，IR 是多周期 CPU 中保持指令稳定的关键模块。

4.1.4.2 接口信号说明

信号名	方向	位宽	描述
IRWrite	input	1	IR 写使能
in_ins	input	32	IM 输出指令
out_ins	output	32	锁存后的当前指令

表 4-7 接口信号说明

4.1.4.3 IRWrite 时序说明

IRWrite 只在 IF 状态有效。在 IF 状态下，IRWrite=1，表示将当前 IM 输出的 in_ins 写入 IR，并形成稳定的 out_ins。在其他状态下，IRWrite=0，IR 保持原有指令不变。

当前状态机包含 IT、IF、DAE、MEM、WB 五个状态，其中 IT 主要用于初始取指准备，DAE 为译码与执行合并状态。

状态	IRWrite	说明
IT	0	初始化取指准备状态，不锁存指令
IF	1	将 IM 输出的 in_ins 写入 IR，形成当前执行指令 out_ins
DAE	0	IR 保持不变，后续译码和执行均基于稳定的 out_ins
MEM	0	IR 保持不变，供 lw/sw 等访存指令继续使用当前指令信息
WB	0	IR 保持不变，直到下一次进入 IF 状态再锁存新指令

表 4-8 时序说明

4.1.4.4 IRWrite 设计思路详解

IRWrite 的设计思路是：只在 IF 状态更新当前指令，后续状态保持指令稳定。当状态机进入 IF 状态时，IM 已经根据当前 PC 输出了指令 in_ins，此时 ControlUnit 置 IRWrite=1，使 IR 在时钟沿将 in_ins 锁存为 out_ins。后续 DAE、MEM 和 WB 状态都围绕这个 out_ins 完成指令执行，不会因为 PC 提前更新或指令存储器输出变化而影响当前指令。

当前设计中普通顺序指令可能会在 IF 阶段提前更新 PC，后续 DAE 阶段也可能通过 InsMemRW 读取下一条顺序指令。因此，IR 的锁存作用更加重要：它保证当前指令一旦在 IF 阶段被写入，就能在后续多个状态中保持不变，使 opcode、Funct3、Funct7、寄存器地址和立即数字段都稳定可用。

如果 IRWrite 在 DAE、MEM 或 WB 等状态中也被置为 1，就可能导致当前正在执行的指令被新指令覆盖，进而造成控制信号、寄存器地址或立即数解析错误。因此，IRWrite 只在 IF 状态有效，是保证多周期 CPU 正确执行的重要设计。

4.2 译码与寄存器读取相关模块设计

4.2.1 RF 寄存器堆模块

4.2.1.1 模块功能描述

RF 模块是通用寄存器堆，用于保存 CPU 运算过程中使用的通用数据。它提供两个读端口和一个写端口，读地址分别为 rs1 和 rs2，写地址为 WR，写数据为 WD。

读出的两个操作数 RD1 和 RD2 一方面送入 U_A 和 U_B 寄存器进行锁存，另一方面在当前版本中会直接作为 ALU 输入选择器的主要数据来源。其中 RD1 送入 MUX_2to1_A，RD2 送入 MUX_3to1_B；U_B 输出的 RD2_r 主要用于 sw 指令在 MEM 阶段向数据存储器写入数据。

4.2.1.2 接口信号说明

信号名	方向	位宽	描述
RR1	input	5	读寄存器地址 1
RR2	input	5	读寄存器地址 2
WR	input	5	写寄存器地址
WD	input	32	写数据
clk	input	1	时钟
RFWrite	input	1	写使能
RD1	output	32	读数据 1

信号名	方向	位宽	描述
RD2	output	32	读数据 2

表 4-9 接口信号说明

4.2.1.3 RFWrite 时序说明

RFWrite 用于控制寄存器堆是否写入数据。R-type、I-type 和 lw 指令在 WB 阶段写回；而 jal/jalr 指令的返回地址写回发生在 DAE 阶段。

指令类型	RFWrite 有效阶段	写回内容
R-type	WB	ALU_result_r
I-type	WB	ALU_result_r
lw	WB	DR_out
jal/jalr	DAE	PCA4
sw	无	不写回
beq/bne	无	不写回

表 4-10 时序说明

4.2.1.4 RFWrite 设计思路详解

RFWrite 的设计思路是：只有当指令的最终结果已经确定时，才允许写入寄存器堆。R-type 和 I-type 指令的结果来自 ALU，其结果在 DAE 阶段产生，并由 U_ALUOut 锁存为 ALU_result_r，随后在 WB 阶段写回寄存器堆。lw 指令的写回数据来自数据存储器读出的 DR_out，也在 WB 阶段写回。jal/jalr 指令的写回数据为 PCA4，即 PC+4，jal/jalr 的写回并不放在 WB 阶段，而是在 DAE 阶段由 ControlUnit 置 RFWrite=1，并通过 WDSel 选择 PCA4 写回寄存器堆。

sw 指令只写数据存储器，不修改寄存器堆；beq/bne 只改变 PC，不修改寄存器堆。因此这些指令中 RFWrite 保持为 0。这样可以避免对寄存器堆产生误写，提高执行正确性。

4.2.2 EXT 立即数扩展模块

4.2.2.1 模块功能描述

EXT 模块用于将指令中的立即数或偏移字段扩展为 32 位数据。它接收 Imm12、Offset 和 ExtSel，输出 Imm32。该模块主要为 I-type、lw 和 sw 等需要立即数参与 ALU 运算或地址计算的指令提供操作数。其中，Imm12 主要对应 I-type 立即数字段，Offset 则根据 opcode 由顶层逻辑选择生成，可用于 sw 指令的 S-type 立即数或其他偏移字段。

4.2.2.2 接口信号说明

信号名	方向	位宽	描述
imm_in	input	12	I-type 立即数字段，当前顶层连接为 Imm12
sw_imm_in	input	12	S-type 或其他偏移字段，当前顶层连接为 Offset
ExtSel	input	1	扩展方式选择信号，用于选择符号扩展或零扩展
imm_out	output	32	扩展后的 32 位立即数，输出为 Imm32

表 4-11 接口信号说明

4.2.2.3 ExtSel 时序说明

ExtSel 主要在 DAE 阶段影响 ALU 输入，因为当前版本中 DAE 状态合并了承担译码与执行功能，立即数经 EXT 扩展为 Imm32 后，会通过 MUX_3to1_B 送入 ALU 的 B 端。

指令类型	ExtSel	说明
addi	SIGNED	立即数作为有符号数参与加法运算
ori	ZERO	立即数作为逻辑操作数参与按位或运算，采用零扩展
lw	SIGNED	基址偏移为有符号数，用于计算访存地址
sw	SIGNED	S-type 偏移为有符号数，用于计算访存地址
R-type	默认值	不使用立即数参与 ALU 运算
Branch	默认值或不依赖 EXT	分支偏移主要由 Offset 和 NPC 相关逻辑处理，通常不通过 Imm32 作为 ALU 主操作数
jalr	SIGNED	立即数作为寄存器间接跳转偏移，与 RD1 相加生成 rs_target

表 4-12 时序说明

4.2.2.4 ExtSel 设计思路详解

不同指令对立即数的解释方式不同。算术和访存类指令通常需要将立即数作为有符号数使用，因此采用符号扩展；逻辑类立即数指令如 `ori` 更适合采用零扩展。`ExtSel` 由 `ControlUnit` 在 `DAE` 状态根据指令类型和 `Funct3` 产生，其中 `ori` 对应零扩展，`addi`、`lw` 和 `sw` 等指令默认采用符号扩展。将扩展方式交给 `ExtSel` 控制，可以让 `EXT` 模块本身保持简单，只需要根据控制信号完成扩展，而不需要直接判断完整指令类型。

4.3 执行通路模块设计

4.3.1 操作数寄存器 `U_A`、`U_B`

4.3.1.1 模块功能描述

`U_A` 和 `U_B` 是两个 32 位寄存器，用于锁存 `RF` 输出的 `RD1` 和 `RD2`，锁存后的结果分别为 `RD1_r` 和 `RD2_r`。`ALU` 输入选择器的主要输入直接来自 `RD1` 和 `RD2`，而不是 `RD1_r` 和 `RD2_r`；其中 `RD1_r` 主要作为辅助锁存或保留路径，`RD2_r` 则主要用于 `sw` 指令在 `MEM` 阶段向 `DM` 提供写入数据。

4.3.1.2 锁存时序说明

`U_A` 和 `U_B` 在时钟沿对 `RF` 的输出 `RD1` 和 `RD2` 进行锁存。由于当前状态机采用 `IT`、`IF`、`DAE`、`MEM`、`WB` 的控制流程，寄存器堆读出的数据可能会在后续状态中继续使用，因此保留中间锁存寄存器可以提高数据保持的稳定性。`RD2_r` 在 `sw` 指令的 `MEM` 阶段作为数据存储器写数据使用；而 `ALU` 在 `DAE` 阶段的主要操作数仍然直接来自 `RD1` 和 `RD2`。

4.3.1.3 设计思路详解

`U_A` 和 `U_B` 的作用是为寄存器堆读出的源操作数提供锁存路径，避免在多周期执行过程中由于读地址变化而导致部分后续阶段使用的数据不稳定。`ALU` 在 `DAE` 阶段使用的主要操作数直接来自 `RD1` 和 `RD2`，而不是 `RD1_r` 和 `RD2_r`。因此，`U_A` 和 `U_B` 不再作为 `ALU` 主输入前的必经寄存器，其中 `U_A` 输出的 `RD1_r` 主要作为保留路径，`U_B` 输出的 `RD2_r` 则用于 `sw` 指令在 `MEM` 阶段向 `DM` 写入数据。这样既保留了多周期结构中的数据锁存能力，也与当前顶层数据通路连接保持一致。

4.3.2 `ALU` 输入 `A` 选择器 `MUX_2to1_A`

4.3.2.1 模块功能描述

`MUX_2to1_A` 用于选择 `ALU` 输入端 `A` 的来源。当前顶层连接中，该模块输入为 `RD1` 和常

数 0，其中常数输入在代码中写作 5'h0，输出为 A，控制信号为 ALUSrcA。

4.3.2.2 ALUSrcA 时序说明

ALUSrcA 主要在 DAE 阶段起作用。当前设计中 DAE 状态合并了承担译码与执行功能，ControlUnit 在该阶段根据指令类型设置 ALUSrcA，从而决定 ALU 的 A 端输入来源。

指令类型	ALUSrcA 选择	说明
R-type	RD1	寄存器-寄存器运算，A 端选择源寄存器 1 的读出数据
I-type	RD1	寄存器-立即数运算，A 端选择源寄存器 1 的读出数据
lw/sw	RD1	以 rs1 作为基址寄存器，用于计算访存地址
beq/bne	RD1	作为比较操作数 1，与 RD2 进行减法比较
jlr	RD1	作为寄存器间接跳转的基址，与立即数相加生成跳转目标

表 4-13 时序说明

4.3.2.3 ALUSrcA 设计思路详解

当前大多数需要使用 ALU 的指令都会选择 RD1 作为 ALU A 端输入。对于 R-type 指令，RD1 与 RD2 共同参与寄存器-寄存器运算；对于 I-type、lw、sw 和 jlr 等指令，RD1 通常作为基址或源操作数，与 Imm32 配合完成立即数运算、地址计算或跳转目标计算；对于 beq/bne 指令，RD1 作为第一个比较操作数，与 RD2 进行减法比较。

4.3.3 ALU 输入 B 选择器 MUX_3to1_B

4.3.3.1 模块功能描述

MUX_3to1_B 用于选择 ALU 输入端 B 的来源。当前顶层连接中，B 端可以选择 RD2、Imm32 或 Offset，输出为 B，控制信号为 ALUSrcB。

4.3.3.2 ALUSrcB 时序说明

ALUSrcB 同样主要在 DAE 阶段有效。当前设计中 DAE 状态合并了承担译码与执行功能，ControlUnit 在该阶段根据指令类型设置 ALUSrcB，从而决定 ALU 的 B 端输入来源。

指令类型	B 端来源	作用
R-type	RD2	两个寄存器操作数参与运算
I-type	Imm32	寄存器与立即数参与运算
lw	Imm32	与 RD1 基址相加，计算访存地址
sw	Imm32	与 RD1 基址相加，计算访存地址
beq/bne	RD2	与 RD1 进行减法比较，产生 zero 信号

指令类型	B 端来源	作用
jalr	Imm32	与 RD1 相加，生成寄存器间接跳转目标

表 4-14 时序说明

4.3.3.3 ALUSrcB 设计思路详解

通过 ALUSrcB 控制 B 端输入，可以让同一个 ALU 支持寄存器运算、立即数运算、访存地址计算、分支比较和 jalr 跳转目标计算。对于 R-type 指令，ALU 的 B 端选择 RD2，与 A 端的 RD1 一起完成寄存器-寄存器运算；对于 I-type 指令，B 端选择 Imm32，与 RD1 完成寄存器-立即数运算；对于 lw 和 sw 指令，B 端同样选择 Imm32，与 RD1 基址相加，得到数据存储器访问地址；对于 beq/bne 指令，B 端选择 RD2，使 ALU 执行 RD1 - RD2，并通过 zero 信号判断两个寄存器是否相等；对于 jalr 指令，B 端选择 Imm32，与 RD1 相加生成寄存器间接跳转目标。当前顶层代码中，MUX_3to1_B 的输入为 RD2、Imm32 和 Offset，其中 RD2 是 ALU B 端的寄存器操作数主输入，Imm32 用于立即数运算和地址计算，Offset 则作为偏移量输入保留。

4.3.4 ALU 算术逻辑单元

4.3.4.1 模块功能描述

ALU 是 CPU 的核心执行单元，用于完成算术运算、逻辑运算、移位运算、地址计算和分支比较。输入为 A、B 和 ALUOp，输出为 ALU_result 和 zero。

4.3.4.2 ALUOp 时序说明

ALUOp 主要在 DAE 阶段有效。当前设计中 DAE 状态合并了承担译码与执行功能，ControlUnit 在该阶段根据 opcode、Funct3 和 Funct7 设置 ALUOp，使 ALU 完成对应的运算、地址计算或分支比较。

指令	ALUOp	功能
add	ADD	加法
sub	SUB	减法
and	AND	按位与
or	OR	按位或
xor	XOR	按位异或
sll	SLL	逻辑左移
srl	SRL	逻辑右移
sra	SRA	算术右移

指令	ALUOp	功能
addi	ADD	寄存器加立即数
ori	OR	寄存器或立即数
lw/sw	ADD	计算访存地址
beq/bne	SUB	比较两个寄存器
jalr	ADD	计算 $rs1 + imm$ ，生成寄存器间接跳转目标

表 4-15 时序说明

4.3.4.3 ALUOp 设计思路详解

ALUOp 由 ControlUnit 根据 opcode、Funct3 和 Funct7 生成。对于 R-type 指令，Funct3 和 Funct7 决定具体运算，例如通过 Funct7[5] 区分 add/sub 和 srl/sra；对于 I-type 指令，Funct3 决定是 addi 还是 ori；对于 lw/sw 指令，ALUOp 固定为 ADD，用于完成基址加偏移的访存地址计算；对于 beq/bne 指令，ALUOp 固定为 SUB，用于比较两个寄存器是否相等；对于 jalr 指令，ALUOp 同样设置为 ADD，用于计算 $rs1 + imm$ 形式的寄存器间接跳转目标。

ALU 的 zero 输出主要用于分支判断。当 beq/bne 指令进入 DAE 阶段时，ControlUnit 设置 ALUSrcA 选择 RD1、ALUSrcB 选择 RD2，并令 ALUOp 为 SUB，使 ALU 执行 $RD1 - RD2$ 。如果结果为 0，则 $zero=1$ ，表示两个寄存器相等；否则 $zero=0$ 。ControlUnit 再根据 zero 和 Funct3 判断 beq 或 bne 是否满足跳转条件，并通过 NPCOp 和 PCWrite 控制 PC 更新。

4.3.5 U_ALUOut 结果寄存器

4.3.5.1 模块功能描述

U_ALUOut 用于锁存 ALU 的输出结果。其输入为 ALU_result，输出为 ALU_result_r。

4.3.5.2 锁存时序说明

ALU 在 DAE 阶段产生运算结果，U_ALUOut 在时钟沿将 ALU_result 锁存为 ALU_result_r。后续 MEM 或 WB 阶段主要使用锁存后的 ALU_result_r，例如 lw/sw 指令在 MEM 阶段使用 $ALU_result_r[11:2]$ 作为数据存储器地址，R-type 和 I-type 指令在 WB 阶段使用 ALU_result_r 作为写回数据。

4.3.5.3 设计思路详解

U_ALUOut 的作用是保存 DAE 阶段的 ALU 执行结果，使该结果能够在后续 MEM 或 WB 阶段稳定使用。对于 R-type 和 I-type 指令，ALU_result_r 在 WB 阶段作为写回数据；对于 lw/sw 指令，ALU_result_r 在 MEM 阶段作为数据存储器访问地址，其中实际送入 DM

的地址为 ALU_result_r[11:2]。通过该寄存器，DAE 阶段的组合运算结果与后续访存、写回阶段实现了解耦。

4.4 访存通路模块设计

4.4.1 DM 数据存储器模块

4.4.1.1 模块功能描述

DM 模块用于实现数据存储器访问，主要服务于 lw 和 sw 指令。lw 指令从 DM 中读取数据，sw 指令将寄存器中的数据写入 DM。

4.4.1.2 接口信号说明

信号名	方向	位宽	描述
Addr	input	10	数据存储器地址，来自 ALU_result_r[11:2]
WD	input	32	写入数据，来自 RD2_r
DMCtrl	input	1	读写控制
clk	input	1	时钟
RD	output	32	读出数据

表 4-16 接口信号说明

4.4.1.3 DMCtrl 时序说明

DMCtrl 只在 MEM 阶段对 lw/sw 指令起作用。

指令	有效阶段	作用
lw	MEM	控制 DM 读出数据
sw	MEM	控制 DM 写入数据
其他指令	无	不访问 DM

表 4-17 时序说明

4.4.1.4 DMCtrl 设计思路详解

lw 和 sw 的有效地址都在 DAE 阶段由 ALU 计算，计算结果随后由 U_ALUOut 锁存为 ALU_result_r。进入 MEM 阶段后，DM 使用 ALU_result_r[11:2] 作为访问地址。对于 sw 指令，写入数据来自 RD2_r，即源寄存器 rs2 的锁存值；对于 lw 指令，数据存储器的读出

数据为 RD。

4.5 写回通路模块设计

4.5.1 写回地址选择器 MUX_3to1

4.5.1.1 模块功能描述

MUX_3to1 用于选择寄存器堆的写地址 WR。当前顶层连接中，三个输入分别为 rd、5'd0 和 5'd31。

4.5.1.2 RegSel 时序说明

RegSel 主要在寄存器堆写回时起作用，只有当 RFWrite 有效时，MUX_3to1 选择出的 WR 才会真正作为寄存器堆写地址使用。

选择来源	写地址	用途
rd	指令中的 rd	普通 R-type、I-type、lw 写回地址；当前 jal/jalr 在未额外修改 RegSel 时也默认使用 rd 作为链接寄存器
5'd0	x0	无效写回或保留输入
5'd31	x31	链接寄存器写回的保留输入，只有在 ControlUnit 明确设置 RegSel 选择该路时才会使用

表 4-18 时序说明

4.5.1.3 RegSel 设计思路详解

RegSel 将写回地址选择从 RF 模块中独立出来。RF 只负责在 RFWrite 有效时根据 WR 写入寄存器，不需要直接关心当前指令类型。当前顶层中，MUX_3to1 提供了 rd、x0 和 x31 三类写地址来源，其中 rd 是主要写回地址，x0 和 x31 更多作为保留或扩展输入。

4.5.2 写回数据选择器 MUX_3to1_LMD

4.5.2.1 模块功能描述

MUX_3to1_LMD 用于选择写回寄存器堆的数据 WD。当前输入为 ALU_result_r、DR_out 和 PCA4，控制信号为 WDSel。

4.5.2.2 WDSel 时序说明

WDSel 主要在寄存器堆写回时有效。当前设计中，R-type、I-type 和 lw 指令在 WB 阶段通过 WDSel 选择写回数据；jal/jalr 指令则在 DAE 阶段通过 WDSel 选择 PCA4 作为返回地址写回。

指令类型	WDSel 选择	写回数据	有效阶段
R-type	ALU_result_r	ALU 运算结果	WB
I-type	ALU_result_r	ALU 运算结果	WB
lw	DR_out	数据存储器读出数据	WB
jal/jalr	PCA4	返回地址 PC+4	DAE

表 4-19 时序说明

4.5.2.3 WDSel 设计思路详解

不同指令写回的数据来源不同。R-type 和 I-type 指令写回 ALU 运算结果，数据来源 ALU_result_r；lw 指令写回数据存储器读出的数据，数据来源为 DR_out；jal/jalr 指令写回返回地址，数据来源为 PCA4。通过 WDSel 将 ALU_result_r、DR_out 和 PCA4 统一选择到 RF 的 WD 端口，可以使寄存器堆写回结构保持统一。

5 ControlUnit 控制单元详细设计

5.1 ControlUnit 模块功能概述

ControlUnit 是 CPU 的控制核心。它根据当前状态、指令字段和 ALU 的 zero 标志，产生控制数据通路运行所需的全部控制信号。该模块采用有限状态机设计，包括状态寄存器、下一状态转移逻辑和控制信号输出逻辑三部分。

5.2 ControlUnit 输入输出接口说明

信号名	方向	位宽	功能
clk	input	1	状态机时钟
rst	input	1	状态机复位
zero	input	1	ALU 比较结果，主要用于 beq/bne 分支判断
in_ins	input	7	来自 IM 输出指令的低 7 位，用于 IF 阶段提前判断指令类型
opcode	input	7	来自 IR 锁存指令 out_ins[6:0]，用于判断当前正在执行的指令类型
Funct7	input	7	功能字段，用于区分部分 R-type 运算，如 add/sub、srl/sra
Funct3	input	3	功能字段，用于区分具体 ALU 运算、分支类型或立即数扩展方式
RFWrite	output	1	寄存器堆写使能
DMCtrl	output	1	数据存储器读写控制

信号名	方向	位宽	功能
PCWrite	output	1	PC 写使能
IRWrite	output	1	IR 写使能
InsMemRW	output	1	指令存储器访问控制
ExtSel	output	1	立即数扩展方式选择
ALUOp	output	4	ALU 运算控制
NPCOp	output	2	下一 PC 来源选择
ALUSrcA	output	1	ALU A 端输入选择
ALUSrcB	output	2	ALU B 端输入选择
RegSel	output	2	写回地址选择
WDSel	output	2	写回数据选择

表 5-1 接口说明

5.3 有限状态机设计

5.3.1 状态定义

当前设计中状态机包含五个状态：

状态	编码	功能
IT	3'b101	初始化取指准备状态，复位后首先进入该状态，用于启动初始指令读取
IF	3'b000	取指与指令锁存状态，将 IM 输出的 in_ins 写入 IR，同时对普通顺序指令提前更新 PC
DAE	3'b001	译码与执行合并状态，完成指令识别、控制信号生成、ALU 运算、地址计算、分支判断和跳转控制
MEM	3'b011	访存状态，只对 lw/sw 指令有效，用于访问数据存储器 DM
WB	3'b010	写回或取指准备状态，用于普通指令和 lw 的写回，也用于分支和跳转类指令后的取指准备

表 5-2 状态说明

5.3.2 状态转移逻辑

状态转移关系如下：

当前状态	条件	下一状态
IT	固定	IF
IF	固定	DAE
DAE	当前指令为 lw/sw	MEM
DAE	其他指令，包括 R-type、I-type、beq/bne、jal/jalr	WB
MEM	当前指令为 sw	IF
MEM	当前指令为 lw	WB
WB	固定	IF

表 5-3 状态逻辑

5.3.3 控制信号默认值设计

ControlUnit 在每次组合逻辑输出控制信号前，都会先给所有控制信号赋默认值。这样做的目的是避免组合逻辑中出现不完整赋值，从而防止综合工具推断出锁存器。

默认值一般表示“不执行写操作、不访问数据存储器、选择普通路径”。当状态机进入某个具体状态后，再根据指令类型覆盖相关控制信号。这样控制逻辑更安全，也更便于调试。

5.4 各状态控制信号生成详解

5.4.1 IT 状态控制信号

IT 状态是复位后的初始化取指准备状态。状态机复位后首先进入 IT，该状态主要用于启动第一次指令读取，为下一周期进入 IF 状态锁存指令做准备。

控制信号	取值	功能
InsMemRW	1	启动指令存储器访问，读取初始指令
其他信号	默认值	不写 PC、不写 IR、不写 RF、不访问 DM

表 5-4 信号说明

IT 状态本身不负责将指令写入 IR，而是先让 IM 输出有效的 in_ins。下一周期进入 IF 状态后，再通过 IRWrite=1 将 in_ins 锁存为 out_ins。

5.4.2 IF 状态控制信号

IF 状态主要完成指令锁存，并根据当前 IM 输出指令的低 7 位 in_ins[6:0] 判断是否可以提前更新 PC。在当前版本中，IF 状态并不是单纯地置 InsMemRW=1，而是重点负责 IRWrite 和

普通顺序指令的 PCWrite 控制。

控制信号	取值	功能
IRWrite	1	将 IM 输出的 in_ins 写入 IR，形成当前执行指令 out_ins
PCWrite	视 in_ins[6:0] 而定	若当前指令为 R-type、I-type、lw、sw 等普通顺序指令，则提前更新 PC
InsMemRW	默认值 0	IF 状态主要锁存已有 in_ins，不单独启动新的取指访问
其他信号	默认值	不写 RF，不访问 DM

表 5-5 信号说明

对于 R-type、I-type、lw 和 sw 指令，下一条地址固定为 PC+4，因此 ControlUnit 在 IF 状态下置 PCWrite=1，使 PC 提前更新到下一条顺序指令地址。对于 beq/bne、jal/jalr 等分支或跳转指令，IF 状态下保持 PCWrite=0，等到 DAE 状态确定目标地址后再更新 PC。

5.4.3 DAE 状态控制信号

DAE 是译码与执行合并状态。该状态根据 opcode、Funct3、Funct7 和 zero 等信号产生不同控制信号，完成 ALU 运算、地址计算、分支判断和跳转控制。

R-type 指令：

控制信号	设置
ALUSrcA	选择 RD1
ALUSrcB	选择 RD2
ALUOp	根据 Funct3/Funct7 设置
InsMemRW	1，用于提前读取下一条顺序指令
RFWrite	0，写回在 WB 状态完成

表 5-6 指令说明

I-type 指令：

控制信号	设置
ALUSrcA	选择 RD1
ALUSrcB	选择 Imm32
ExtSel	addi 采用符号扩展，ori 采用零扩展
ALUOp	addi 为 ADD，ori 为 OR
InsMemRW	1，用于提前读取下一条顺序指令
RFWrite	0，写回在 WB 状态完成

表 5-7 指令说明

lw/sw 指令：

控制信号	设置
ALUSrcA	选择 RD1
ALUSrcB	选择 Imm32
ALUOp	ADD，用于计算访存地址
InsMemRW	1，用于提前读取下一条顺序指令
RFWrite	0

表 5-8 指令说明

lw/sw 指令在 DAE 状态下只完成访存地址计算，计算结果随后由 U_ALUOut 锁存为 ALU_result_r，供 MEM 状态访问数据存储器使用。

beq/bne 指令：

控制信号	设置
ALUSrcA	选择 RD1
ALUSrcB	选择 RD2
ALUOp	SUB
NPCOp	根据 zero 和 Funct3 判断选择顺序地址或分支目标
PCWrite	1

RFWrite	0
InsMemRW	0

表 5-9 指令说明

jal 指令:

控制信号	设置
NPCOp	选择 J-type 跳转目标
WDSel	选择 PCA4
PCWrite	1
RFWrite	1
InsMemRW	0

表 5-10 指令说明

jalr 指令:

控制信号	设置
ALUSrcA	选择 RD1
ALUSrcB	选择 Imm32
ALUOp	ADD
NPCOp	选择 rs_target
WDSel	选择 PCA4
PCWrite	1
RFWrite	1
InsMemRW	0

表 5-11 指令说明

5.4.4 MEM 状态控制信号

MEM 阶段只对 lw 和 sw 指令有意义。

lw 指令：

控制信号	设置
DMCtrl	读控制
PCWrite	0
RFWrite	0

表 5-12 指令说明

sw 指令：

控制信号	设置
DMCtrl	写控制
RFWrite	0
PCWrite	0

表 5-13 指令说明

5.4.5 WB 状态控制信号

WB 状态主要用于 R-type、I-type 和 lw 指令的寄存器写回；对于 beq/bne、jal/jalr 等分支或跳转类指令，WB 状态不再执行寄存器写回和 PC 更新，主要用于配合后续指令读取准备。

指令类型	RFWrite	WDSel	PCWrite	InsMemRW	说明
R-type	1	ALU_result_r	0	0	写回 ALU 运算结果
I-type	1	ALU_result_r	0	0	写回立即数运算结果
lw	1	DR_out	0	0	写回数据存储器读出数据
sw	不进入 WB	—	—	—	sw 在 MEM 状态结束后直接回到 IF
beq/bne	0	默认值	0	1	不写回寄存器，只用于目标地址确定后的取指准备
jal/jalr	0	默认值	0	1	返回地址已在 DAE 状态写回，WB 状态只用于后续取指准备

表 5-14 控制信号说明

5.5 各重点控制信号时序说明与设计思路

5.5.1 PCWrite

普通顺序指令提前更新 PC，控制转移指令等待目标地址确定后再更新 PC。对于 R-type、I-type、lw 和 sw 等顺序执行指令，下一条指令地址固定为 PC+4，因此可以在 IF 阶段提前更新 PC。由于当前指令已经通过 IRWrite 锁存到 IR 中，后续 DAE、MEM 或 WB 状态仍然基于稳定的 out_ins 执行，所以 PC 的提前更新不会影响当前指令。

对于 beq/bne 指令，是否跳转需要在 DAE 状态通过 ALU 比较结果 zero 来判断；对于 jal 和 jalr 指令，目标地址也需要在 DAE 状态根据 Offset20 或 ALU_result 确定。因此这些指令在 IF 状态不更新 PC，而是在 DAE 状态确定目标后再通过 PCWrite 写入新的 NPC。这样既能保证普通指令取指连续，又能避免分支和跳转指令错误更新 PC。

当前代码中，PCWrite 的有效阶段如下：

指令类型	PCWrite 有效阶段	说明
R-type	IF	普通顺序指令，提前更新 PC 为 PC+4
I-type	IF	普通顺序指令，提前更新 PC 为 PC+4
lw	IF	IF 阶段提前更新 PC，后续完成地址计算、访存和写回
sw	IF	IF 阶段提前更新 PC，后续完成地址计算和写存储器
beq/bne	DAE	在 DAE 阶段根据 zero 和 Funct3 判断是否跳转，再更新 PC
jal	DAE	在 DAE 阶段选择 J-type 跳转目标并更新 PC
jalr	DAE	在 DAE 阶段由 ALU 计算 rs1 + imm，并选择寄存器间接跳转目标更新 PC

表 5-15 时序说明

5.5.2 IRWrite

IRWrite 控制 IR 是否写入新指令。它只在 IF 阶段有效。

状态	IRWrite
IT	0
IF	1
DAE	0
MEM	0

WB	0
----	---

表 5-16 时序说明

设计思路是取指阶段锁存当前指令，后续阶段保持不变，保证一条指令在多个周期中稳定执行。

5.5.3 InsMemRW

InsMemRW 控制 IM 指令存储器的访问。InsMemRW 根据取指准备和指令类型在不同状态下置位。IT 状态用于启动初始取指；普通顺序指令在 DAE 状态置 InsMemRW=1，用于提前读取下一条顺序指令；分支和跳转类指令则在 WB 状态置 InsMemRW=1，用于从更新后的目标 PC 地址处读取下一条指令。

状态/指令情况	InsMemRW	说明
IT	1	启动初始指令读取
IF	0	主要通过 IRWrite 将已有 in_ins 锁存到 IR
DAE，且当前指令为 R-type/I-type/lw/sw	1	普通顺序指令提前读取下一条顺序指令
DAE，且当前指令为 beq/bne/jal/jalr	0	分支或跳转目标尚在确定过程中，不立即读取新指令
MEM	0	访存阶段访问的是 DM，不访问 IM
WB，且当前指令为 beq/bne/jal/jalr	1	PC 已在 DAE 状态更新，用于读取目标地址处的下一条指令
WB，且当前指令为 R-type/I-type/lw	0	普通顺序指令的下一条指令已在前面阶段进行读取准备

表 5-17 时序说明

该信号用于标识当前处于取指阶段，同时为指令存储器接口保留扩展能力。

5.5.4 RFWrite

RFWrite 控制寄存器堆写入。当前版本中，R-type、I-type 和 lw 指令在 WB 阶段写回；jal/jalr 指令在 DAE 阶段写回 PCA4。

指令	RFWrite 有效阶段	写回内容
R-type/I-type	WB	ALU_result_r
lw	WB	DR_out
jal/jalr	DAE	PCA4

sw	无	不写回寄存器
beq/bne	无	不写回寄存器

表 5-18 时序说明

设计思路是只有在最终结果确定后才写回寄存器，避免无效写入或错误写入。

5.5.5 DMCtrl

DMCtrl 控制数据存储器读写，只在 MEM 阶段对 lw/sw 有效。

指令	DMCtrl 作用
lw	读数据
sw	写数据
其他	不访问 DM

表 5-19 时序说明

设计思路是让数据存储器只在访存阶段工作，地址由 DAE 阶段的 ALU 计算产生，并由 U_ALUOut 锁存为 ALU_result_r。进入 MEM 阶段后，DM 使用 ALU_result_r[11:2] 作为访问地址。lw 指令控制 DM 读出数据，sw 指令控制 DM 写入 RD2_r。

5.5.6 ExtSel

ExtSel 控制立即数扩展方式。

指令	ExtSel
addi/lw/sw/jalr	符号扩展
ori	零扩展
R-type	默认值，不使用立即数
beq/bne	默认值，分支偏移主要由 Offset 和 NPC 相关逻辑处理

表 5-20 时序说明

设计思路是将立即数扩展方式从 EXT 模块中抽离出来，由 ControlUnit 根据指令类型和 Funct3 决定。对于 addi、lw、sw 和 jalr 等需要将立即数作为有符号偏移参与加法运算的指

令, ExtSel 选择符号扩展; 对于 ori 这类逻辑立即数指令, ExtSel 选择零扩展。这样 EXT 模块只需要根据 ExtSel 完成扩展, 不需要直接判断完整指令类型, 降低了模块耦合。

5.5.7 ALUOp

ALUOp 控制 ALU 运算类型, 主要在 DAE 阶段有效。DAE 状态合并了承担译码与执行功能, ControlUnit 在该状态下根据 opcode、Funct3 和 Funct7 产生 ALUOp, 使 ALU 完成对应的算术逻辑运算、地址计算、分支比较或 jalr 跳转目标计算。

指令类型	ALUOp
R-type	由 Funct3/Funct7 决定
I-type	由 Funct3 决定
lw/sw	ADD, 用于计算访存地址
beq/bne	SUB, 用于比较两个寄存器是否相等
jalr	ADD, 用于计算 $rs1 + imm$ 跳转目标
jal	默认值, 主要由 NPC 根据 Offset20 生成跳转目标

表 5-21 时序说明

设计思路是复用 ALU 完成算术逻辑运算、地址计算和分支比较, 提高硬件利用率。

5.5.8 NPCOp

NPCOp 控制下一 PC 的来源。

场景	NPCOp 选择
顺序执行	PC+4
分支成立	PC+Offset
jal	J-type 跳转目标
jalr	rs 相关目标

表 5-22 时序说明

NPCOp 必须与 PCWrite 配合使用。对于普通顺序指令, PCWrite 在 IF 阶段有效, 此时 NPCOp 通常保持默认顺序地址选择; 对于 beq/bne、jal 和 jalr 等控制转移指令, PCWrite 在

DAE 阶段有效，此时 NPCOp 根据分支判断或跳转类型选择对应的下一地址来源。

5.5.9 ALUSrcA

ALUSrcA 控制 ALU A 端输入。ALU A 端主要选择 RF 直接输出的 RD1，而不是 U_A 锁存后的 RD1_r。

指令类型	A 输入
R-type	RD1
I-type	RD1
lw/sw	RD1，作为访存基址
beq/bne	RD1，作为比较操作数 1
jalr	RD1，作为寄存器间接跳转基址
其他/保留	5'h0 / 常数 0

表 5-23 时序说明

该信号为 ALU A 端提供选择能力。当前大多数指令都选择 RD1 作为 A 端输入，用于寄存器运算、立即数运算、访存地址计算、分支比较和 jalr 跳转目标计算。MUX_2to1_A 的另一输入为 5'h0，即常数 0，当前主要作为保留输入，为后续扩展特殊运算或地址生成方式提供接口。

5.5.10 ALUSrcB

ALUSrcB 控制 ALU B 端输入。

指令类型	B 输入
R-type	RD2
I-type	Imm32
lw/sw	Imm32
beq/bne	RD2
jalr	Imm32
其他/保留	Offset

表 5-24 信号说明

设计思路是让寄存器操作数、立即数和偏移量都能通过同一个 ALU 参与运算。

5.5.11 RegSel

RegSel 控制写回寄存器地址。

选择	来源
普通写回	rd
无效写回/保留	x0
链接寄存器	x31

表 5-25 信号说明

RegSel 主要在 RFWrite 有效时与寄存器堆写回配合使用。R-type、I-type 和 lw 指令在 WB 阶段写回；jal/jalr 指令在 DAE 阶段写回 PCA4。

5.5.12 WDSel

WDSel 控制写回数据来源。

指令类型	写回数据
R-type/I-type	ALU_result_r
lw	DR_out
jal/jalr	PCA4

表 5-26 信号说明

设计思路是将不同来源的结果统一送入寄存器堆 WD 端口，使 RF 写回接口保持一致。

5.6 控制信号默认值设计

5.6.1 默认值初始化的作用

在 ControlUnit 的控制信号输出逻辑中，所有控制信号都会先被赋一个默认值，然后再根据当前状态和指令类型对部分信号进行覆盖赋值。该设计方式属于组合逻辑控制单元中常用的写法，主要目的是保证在任意状态、任意指令输入条件下，控制信号都有明确取值，避免因为某些分支没有赋值而产生不确定状态。

在本设计中，控制信号默认值如下：

控制信号	默认值含义
RFWrite = 0	默认不写寄存器堆
DMCtrl = 0	默认不执行数据存储器写操作
PCWrite = 0	默认不更新 PC
IRWrite = 0	默认不写入新指令
InsMemRW = 0	默认不进行取指访问
ExtSel = ExtSel_SIGNED	默认采用符号扩展
ALUOp = ALUOp_ADD	默认 ALU 执行加法
NPCOp = NPC_PC	默认下一地址为顺序地址
ALUSrcA = ALUSrcA_A	默认 ALU A 端选择寄存器操作数
ALUSrcB = ALUSrcB_B	默认 ALU B 端选择寄存器操作数
RegSel = RegSel_rd	默认写回地址选择 rd
WDSel = WDSel_FromALU	默认写回数据选择 ALU 结果

表 5-27 信号说明

这些默认值体现了一个基本原则：默认情况下不改变系统状态，只有在明确需要执行写入、访存、跳转、指令读取或选择特殊数据源时，才由具体状态覆盖相关控制信号。例如，RFWrite 默认置 0，可以防止非写回类指令误写寄存器堆；PCWrite 默认置 0，可以防止 PC 在不需要更新时被错误修改；IRWrite 默认置 0，可以保证当前指令在 DAE、MEM、WB 等后续状态中保持稳定；InsMemRW 默认置 0，则可以避免指令存储器在无需求的状态下被重复访问。

5.6.2 防止组合逻辑锁存器的设计原因

ControlUnit 的第三段逻辑是组合逻辑，用于根据 state、opcode、Funct3、Funct7 和 zero 生成控制信号。组合逻辑有一个重要要求：在所有可能的执行路径中，输出信号都必须被赋值。如果某个控制信号只在部分分支中赋值，而在其他分支中没有赋值，综合工具可能会认为该信号需要保持上一次的值，从而推断出 latch。

在 CPU 控制器中，错误推断 latch 会带来明显风险。一方面，控制信号可能在状态切换后仍然保持上一状态的值，例如上一周期 RFWrite=1，如果下一状态没有重新赋值，就可能导致寄存器堆被误写；另一方面，PCWrite、DMCtrl 这类信号如果残留上一周期的值，会直接造成 PC 错误更新或数据存储器错误写入。

因此，本设计在 always @(*) 的开头先对全部控制信号赋默认值，然后再进入 case(state) 判断。这样做可以保证每次组合逻辑重新计算时，所有控制信号都会从一个确定的安全状态开始，再根据当前状态和指令类型进行覆盖。该设计既能避免锁存器推断，也能提高控制逻辑的

稳定性和可读性。

可以概括为：

默认赋值 → 根据当前状态覆盖 → 根据 opcode/Funct3/Funct7 进一步细化

这种写法使控制信号生成逻辑更接近真实硬件控制器的设计思路，所有无关模块在默认情况下保持不动作，只有当前阶段真正需要工作的模块才被控制信号打开。

5.6.3 默认控制信号与各状态覆盖赋值关系

默认控制信号并不是最终控制结果，而是 ControlUnit 在每次组合逻辑输出前给出的基础值。进入不同状态后，控制器只覆盖当前状态需要使用的控制信号，其余信号继续保持默认值。这样可以避免组合逻辑不完整赋值导致锁存器推断，也能保证在没有明确控制需求时，系统不会误写 PC、寄存器堆或数据存储器。

在 IT 状态下，控制器只覆盖 InsMemRW。IT 是复位后的初始化取指准备状态，只在复位后进入一次，用于启动初始指令读取。该状态下 InsMemRW=1，表示使能指令存储器访问；其他控制信号保持默认值，不写 PC、不写 IR、不写寄存器堆，也不访问数据存储器。

在 IF 状态下，控制器主要覆盖 IRWrite，并根据 in_ins[6:0] 判断当前取出的指令是否属于普通顺序指令。IRWrite=1 时，IM 输出的 in_ins 被写入 IR，形成后续执行所使用的 out_ins。对于 R-type、I-type、lw 和 sw 这类普通顺序指令，IF 状态下会置 PCWrite=1，使 PC 提前更新为 PC+4；对于 beq/bne、jal 和 jalr 等控制转移类指令，IF 状态下保持 PCWrite=0，等待 DAE 状态确定分支或跳转目标后再更新 PC。因此，当前版本中的 IF 状态并不是单纯的取指阶段，而是完成指令锁存，并对普通顺序指令提前更新 PC。由于当前指令已经被 IR 锁存为 out_ins，后续 DAE、MEM、WB 状态仍然基于稳定的 out_ins 执行，所以普通指令在 IF 状态提前更新 PC 不会影响当前指令的执行。

在 DAE 状态下，控制器会根据 opcode、Funct3、Funct7 和 zero 覆盖相关控制信号。R-type 指令会覆盖 ALUSrcA、ALUSrcB 和 ALUOp，用于完成寄存器之间的算术逻辑运算；I-type 指令会覆盖 ALUSrcA、ALUSrcB、ExtSel 和 ALUOp，用于完成寄存器与立即数之间的运算；lw/sw 指令会覆盖 ALUSrcA、ALUSrcB 和 ALUOp，用于计算访存地址。对于这些普通顺序指令，DAE 状态还会置 InsMemRW=1，用于提前读取下一条顺序指令。

对于 beq/bne 指令，DAE 状态会覆盖 ALUSrcA、ALUSrcB、ALUOp、NPCOp 和 PCWrite。此时 ALU 执行 RD1 - RD2，并通过 zero 判断两个寄存器是否相等；控制器再根据 Funct3 和 zero 判断分支是否成立，如果条件成立则 NPCOp 选择分支目标地址，否则选择顺序地址，随后通过 PCWrite=1 更新 PC。对于 jal 指令，DAE 状态会覆盖 NPCOp、PCWrite、WDSel

和 RFWrite，用于选择跳转目标、更新 PC，并将 PCA4 写回寄存器堆。对于 jalr 指令，DAE 状态除了覆盖 NPCOp、PCWrite、WDSel 和 RFWrite 外，还会覆盖 ALUSrcA、ALUSrcB 和 ALUOp，使 ALU 计算 $rs1 + imm$ ，并将 ALU_result 作为 rs_target 送入 NPC 生成跳转目标。需要注意的是，jal/jalr 的返回地址写回发生在 DAE 状态，而不是 WB 状态。

在 MEM 状态下，只有 lw 和 sw 会覆盖 DMCtrl。lw 指令将 DMCtrl 设置为读控制，数据存储器根据 ALU_result_r[11:2] 读出数据 RD，并通过 DR_out 送往写回路径；sw 指令将 DMCtrl 设置为写控制，把 RD2_r 写入 DM 对应地址。当前版本中，sw 的 PC 已经在 IF 状态提前更新，因此 MEM 状态不再覆盖 PCWrite 和 NPCOp。

在 WB 状态下，控制器主要覆盖 RFWrite、WDSel 和 InsMemRW。R-type 和 I-type 指令在 WB 状态下置 RFWrite=1，并通过 WDSel=FromALU 选择 ALU_result_r 写回寄存器堆；lw 指令在 WB 状态下置 RFWrite=1，并通过 WDSel=FromMEM 选择 DR_out 写回寄存器堆。对于 beq/bne、jal 和 jalr，WB 状态下不再写回寄存器，RFWrite=0，主要通过 InsMemRW=1 读取更新后 PC 地址处的下一条指令，为下一轮 IF 锁存做准备。

这种“默认值 + 状态覆盖”的方式使控制器结构更加清楚：默认值保证系统处于安全状态，具体状态只打开当前周期需要工作的硬件模块。当前版本的控制特点可以概括为：IT 只用于复位后的初始取指准备；IF 负责锁存指令，并对普通顺序指令提前更新 PC；DAE 合并译码与执行，完成 ALU 运算、地址计算、分支判断和跳转控制；MEM 只服务于 lw/sw 访存；WB 主要服务于 R-type、I-type 和 lw 的写回，以及分支和跳转类指令后的取指准备。

5.7 各状态控制信号生成详解

5.7.1 IT 状态控制信号

IT 状态是复位后的初始化取指准备状态。系统复位后，状态机首先进入 IT，该状态主要用于启动初始指令读取，为下一周期进入 IF 状态锁存指令做准备。

控制信号	取值	作用
InsMemRW	1	使能指令存储器访问，启动初始指令读取
其他控制信号	默认值	不写 PC、不写 IR、不写寄存器堆、不访问数据存储器

表 5-28 信号说明

IT 状态下并不直接将指令写入 IR，而是先让 IM 根据当前 PC 输出有效的 in_ins。随后状态机进入 IF 状态，由 IF 状态通过 IRWrite=1 将 in_ins 锁存为 out_ins。

5.7.2 IF 状态控制信号

IF 状态主要完成指令锁存，同时根据 in_ins[6:0] 判断普通顺序指令是否可以提前更新 PC。当前版本中，IF 状态不再简单理解为“只负责取指”，而是负责将已经从 IM 输出的 in_ins 写入 IR，并对 R-type、I-type、lw、sw 等顺序指令提前执行 PC+4 更新。

控制信号	取值	作用
IRWrite	1	将 in_ins 写入 IR，形成当前执行指令 out_ins
PCWrite	视 in_ins[6:0] 而定	若当前指令为 R-type、I-type、lw、sw，则提前更新 PC
InsMemRW	默认值 0	IF 状态主要锁存已有指令，不单独启动新的指令读取
其他控制信号	默认值	不写寄存器堆，不访问数据存储器

表 5-29 信号说明

对于 R-type、I-type、lw 和 sw 指令，下一条指令地址固定为 PC+4，因此可以在 IF 状态提前更新 PC。对于 beq/bne、jal/jalr 这类可能改变控制流的指令，IF 状态下保持 PCWrite=0，等待 DAE 状态完成分支判断或跳转目标计算后再更新 PC。

5.7.3 DAE 状态控制信号

DAE 状态是译码与执行合并状态，用于代替旧版本中的 ID 和 EX 两个状态。在该状态中，ControlUnit 根据 opcode、Funct3、Funct7 和 zero 等信号产生控制信号，并完成 ALU 运算、访存地址计算、分支判断或跳转控制。

R-type 指令

控制信号	取值思路
ALUSrcA	选择 RD1
ALUSrcB	选择 RD2
ALUOp	根据 Funct3/Funct7 选择 add、sub、and、or、xor、sll、srl、sra
InsMemRW	置 1，用于提前读取下一条顺序指令
RFWrite	0，写回在 WB 状态完成

表 5-30 指令说明

R-type 指令的两个操作数均来自寄存器堆，ALU 的 A、B 端分别直接选择 RD1 和 RD2

I-type 指令

控制信号	取值思路
ALUSrcA	选择 RD1
ALUSrcB	选择 Imm32
ExtSel	addi 选择符号扩展，ori 选择零扩展
ALUOp	addi 对应 ADD，ori 对应 OR
InsMemRW	置 1，用于提前读取下一条顺序指令
RFWrite	0，写回在 WB 状态完成

表 5-31 指令说明

I-type 指令使用一个寄存器操作数和一个立即数操作数，因此 ALU A 端选择 RD1，B 端选择扩展后的立即数 Imm32。

lw/sw 指令

控制信号	取值思路
ALUSrcA	选择 RD1
ALUSrcB	选择 Imm32
ALUOp	ADD
InsMemRW	置 1，用于提前读取下一条顺序指令
RFWrite	0

表 5-32 指令说明

lw 和 sw 在 DAE 状态使用 ALU 计算数据存储器访问地址，即 $RD1 + Imm32$ 。计算结果随后由 U_ALUOut 锁存为 ALU_result_r，供 MEM 状态访问 DM 使用。

beq/bne 指令

控制信号	取值思路
ALUSrcA	选择 RD1
ALUSrcB	选择 RD2
ALUOp	SUB

控制信号	取值思路
NPCOp	根据 zero 和 Funct3 选择顺序地址或分支目标地址
PCWrite	1
RFWrite	0
InsMemRW	0

表 5-33 指令说明

分支指令在 DAE 状态通过 ALU 做减法比较，即执行 $RD1 - RD2$ 。如果 beq 满足 $zero=1$ ，或 bne 满足 $zero=0$ ，则 NPCOp 选择分支目标地址；否则选择顺序地址。随后通过 $PCWrite=1$ 将对应的 NPC 写入 PC。

jal 指令

控制信号	取值思路
NPCOp	选择 J-type 跳转目标
WDSel	选择 PCA4
PCWrite	1
RFWrite	1
InsMemRW	0

表 5-34 指令说明

jal 在 DAE 状态完成跳转目标选择和返回地址写回。NPC 根据 Offset20 生成跳转目标，PCA4 作为返回地址通过写回数据选择器送入寄存器堆。

jalr 指令

控制信号	取值思路
ALUSrcA	选择 RD1
ALUSrcB	选择 Imm32
ALUOp	ADD
NPCOp	选择 rs_target

控制信号	取值思路
WDSel	选择 PCA4
PCWrite	1
RFWrite	1
InsMemRW	0

表 5-35 指令说明

jalr 在 DAE 状态由 ALU 计算 $rs1 + imm$ ，其结果 ALU_result 直接送入 NPC 的 rs_target 端口，用于生成寄存器间接跳转目标。同时，PCA4 作为返回地址写回寄存器堆。

5.7.4 MEM 状态控制信号

MEM 状态用于 lw 和 sw 指令访问数据存储器。

对于 lw 指令：

控制信号	作用
DMCtrl	控制 DM 读出数据
PCWrite	0
RFWrite	0

表 5-36 指令说明

lw 在 MEM 状态使用 ALU_result_r[11:2] 作为数据存储器地址，从 DM 中读取数据。读出的数据为 RD，当前顶层中通过 $assign\ DR_out = RD$ 直接送往写回选择器，但此时还不写入寄存器堆，真正写回在 WB 状态完成。

对于 sw 指令：

控制信号	作用
DMCtrl	控制 DM 写入数据
PCWrite	0
RFWrite	0

表 5-37 指令说明

sw 指令在 MEM 状态使用 ALU_result_r[11:2] 作为地址，将 RD2_r 写入数据存储器。由于 sw 属于普通顺序指令，PC 已经在 IF 状态提前更新为 PC+4，因此 MEM 状态不再需要置 PCWrite=1。

5.7.5 WB 状态控制信号

WB 状态主要用于 R-type、I-type 和 lw 指令的寄存器写回；对于 beq/bne、jal/jalr 等分支和跳转类指令，WB 状态不再执行寄存器写回和 PC 更新，主要用于配合后续指令读取准备。

指令类型	RFWrite	WDSel	PCWrite	InsMemRW	说明
R-type	1	ALU_result_r	0	0	写回 ALU 运算结果
I-type	1	ALU_result_r	0	0	写回立即数运算结果
lw	1	DR_out	0	0	写回数据存储器读出数据
sw	不进入 WB	—	—	—	sw 在 MEM 状态结束后直接回到 IF
beq/bne	0	默认值	0	1	不写回寄存器，只用于目标地址确定后的取指准备
jal/jalr	0	默认值	0	1	返回地址已在 DAE 状态写回，WB 状态只用于后续取指准备

表 5-38 指令说明

R-type、I-type 和 lw 在 WB 状态完成写回；sw 不进入 WB；beq/bne、jal/jalr 虽然会进入 WB，但主要用于配合 InsMemRW 读取下一条指令。其中，jal/jalr 的 PCA4 写回已经在 DAE 状态完成。

5.8 各指令控制流程

5.8.1 R-type 指令控制流程

R-type 指令包括 add、sub、and、or、xor、sll、srl、sra 等，主要完成两个寄存器之间的算术逻辑或移位运算。其状态执行路径为：

IF → DAE → WB → IF

在 IF 状态下，ControlUnit 令 IRWrite=1，将 IM 输出的 in_ins 锁存到 IR 中，形成后续执行使用的 out_ins。由于 R-type 属于普通顺序执行指令，控制器会根据 in_ins[6:0] 判断当前指令类型，并置 PCWrite=1，使 PC 提前更新为 PC+4。

在 DAE 状态下，顶层组合逻辑从 out_ins 中解析出 opcode、Funct3、Funct7、rs1、rs2 和 rd

等字段。寄存器堆 RF 根据 rs1 和 rs2 读出 RD1 和 RD2。ControlUnit 令 ALUSrcA 选择 RD1, ALUSrcB 选择 RD2。ALUOp 由 Funct3 和 Funct7[5] 决定, 例如 Funct3=000 时通过 Funct7[5] 区分 add 和 sub, Funct3=101 时通过 Funct7[5] 区分 srl 和 sra。ALU 运算结果在该状态产生, 并由 U_ALUOut 锁存为 ALU_result_r。

在 WB 状态下, ControlUnit 令 RFWrite=1, WDSel 选择 ALU_result_r, RegSel 默认选择 rd, 于是 ALU 运算结果通过写回数据选择器送到 RF 的 WD 端口, 并在时钟沿写入目标寄存器。由于 R-type 指令的 PC 已经在 IF 状态提前更新, 所以 WB 状态下不再需要置 PCWrite=1。

5.8.2 I-type 指令控制流程

I-type 指令主要包括 addi 和 ori, 用于完成寄存器与立即数之间的运算。其状态执行路径为:
IF → DAE → WB → IF

在 IF 状态下, CPU 将 IM 输出的指令锁存到 IR 中。由于 I-type 也属于普通顺序执行指令, PCWrite 在 IF 状态有效, PC 提前更新为 PC+4。

在 DAE 状态下, 顶层逻辑解析 rs1、rd、Imm12、opcode 和 Funct3。RF 根据 rs1 读出源操作数 RD1, EXT 根据 Imm12 和 ExtSel 生成 Imm32。当指令为 addi 时, ExtSel 采用符号扩展, ALUOp 设置为 ADD; 当指令为 ori 时, ExtSel 设置为零扩展, ALUOp 设置为 OR。此时 ALUSrcA 选择 RD1, ALUSrcB 选择 Imm32, ALU 完成寄存器与立即数之间的运算, 结果由 U_ALUOut 锁存为 ALU_result_r。

在 WB 状态下, RFWrite=1, WDSel 选择 ALU_result_r, RegSel 选择 rd, 运算结果写回寄存器堆。此时 PCWrite=0, 因为顺序 PC 已经在 IF 状态更新。

5.8.3 lw 指令控制流程

lw 指令用于从数据存储器中读取一个字, 并写回寄存器堆。其状态执行路径为:

IF → DAE → MEM → WB → IF

在 IF 状态下, CPU 将 lw 指令锁存到 IR 中。由于 lw 是普通顺序执行指令, PCWrite 在 IF 状态有效, PC 提前更新为 PC+4。

在 DAE 状态下, 顶层逻辑解析 rs1、rd 和立即数字段。RF 根据 rs1 读出基址寄存器值 RD1, EXT 将立即数扩展为 Imm32。ControlUnit 设置 ALUSrcA 选择 RD1, ALUSrcB 选择 Imm32, ALUOp 设置为 ADD。ALU 完成有效地址计算:

有效地址 = RD1 + Imm32 该地址结果由 U_ALUOut 锁存为 ALU_result_r。

在 MEM 状态下, ControlUnit 将 DMCtrl 设置为读控制。DM 使用 ALU_result_r[11:2] 作为访问地址, 读出数据 RD。当前顶层代码中没有使用单独的 U_DR 寄存器, 而是通过: assign DR_out = RD;使 DR_out 直接等于数据存储器读出数据。

在 WB 状态下, RFWrite=1, WDSel 选择 DR_out, RegSel 选择 rd, 从数据存储器读出的数据写回寄存器堆。此时 PCWrite=0, 因为 lw 的顺序 PC 已经在 IF 状态更新。

5.8.4 sw 指令控制流程

sw 指令用于将寄存器中的数据写入数据存储器。其状态执行路径为:

IF → DAE → MEM → IF

在 IF 状态下, CPU 将 sw 指令锁存到 IR 中。由于 sw 是普通顺序执行指令, PCWrite 在 IF 状态有效, PC 提前更新为 PC+4。

在 DAE 状态下, 顶层逻辑解析 rs1 和 rs2, 其中 rs1 对应基址寄存器, rs2 对应需要写入存储器的数据寄存器。同时, S-type 立即数字段通过 Offset 逻辑拼接得到, 并送入 EXT 生成地址偏移。ControlUnit 设置 ALUSrcA 选择 RD1, ALUSrcB 选择 Imm32, ALUOp 设置为 ADD。ALU 完成数据存储器写地址计算:

有效地址 = RD1 + Imm32

该地址结果被 U_ALUOut 锁存为 ALU_result_r。

在 MEM 状态下, ControlUnit 将 DMCtrl 设置为写控制。DM 使用 ALU_result_r[11:2] 作为地址, 将 RD2_r 写入数据存储器。由于 sw 指令不需要写回寄存器堆, 因此没有 WB 阶段, RFWrite 始终保持为 0。需要注意的是, sw 的 PC 已经在 IF 状态提前更新, 所以 MEM 状态不再需要置 PCWrite=1。

5.8.5 beq 指令控制流程

beq 指令用于判断两个寄存器是否相等, 若相等则发生分支跳转。其状态执行路径为:

IF → DAE → WB → IF

在 IF 状态下，CPU 将 beq 指令锁存到 IR 中。由于 beq 是控制转移类指令，不能在目标地址尚未确定时提前更新 PC，因此 IF 状态下 PCWrite=0。

在 DAE 状态下，顶层逻辑解析 rs1、rs2、Funct3 和 B-type 偏移字段。RF 读出两个待比较的寄存器值 RD1 和 RD2。ControlUnit 设置 ALUSrcA 选择 RD1，ALUSrcB 选择 RD2，ALUOp 设置为 SUB，使 ALU 执行：

RD1 - RD2

如果两个寄存器相等，则 ALU 结果为 0，zero=1。对于 beq 指令，当 Funct3 表示 beq 且 zero=1 时，ControlUnit 将 NPCOp 设置为分支目标地址选择；如果 zero=0，则 NPCOp 选择顺序地址。无论分支是否成立，beq 都在 DAE 状态完成 PC 更新，因此 PCWrite=1。

beq 不需要访问数据存储器，也不需要写回寄存器堆，因此 DMCtrl 保持默认值，RFWrite=0。随后状态机进入 WB 状态，该状态不进行寄存器写回，主要用于通过 InsMemRW=1 读取更新后的 PC 地址处的下一条指令。

5.8.6 bne 指令控制流程

bne 指令用于判断两个寄存器是否不相等，若不相等则发生分支跳转。其状态执行路径为：

IF → DAE → WB → IF

在 IF 状态下，CPU 将 bne 指令锁存到 IR 中。由于 bne 也属于控制转移类指令，IF 状态下不提前更新 PC。

在 DAE 状态下，RF 读出 RD1 和 RD2，ALU 执行：

RD1 - RD2

并根据结果生成 zero。与 beq 不同的是，bne 的跳转条件与 beq 相反：当 zero=0 时，说明两个寄存器不相等，bne 条件成立，NPCOp 选择分支目标地址；当 zero=1 时，说明两个寄存器相等，bne 条件不成立，NPCOp 选择顺序地址。DAE 状态下 PCWrite=1，完成 PC 更新。

bne 不访问数据存储器，也不写回寄存器堆，因此 DMCtrl 保持默认值，RFWrite=0。随后进入 WB 状态，主要用于配合 InsMemRW 完成下一条指令读取准备。

5.8.7 jal 指令控制流程

jal 指令是无条件跳转并链接指令，用于跳转到目标地址，同时保存返回地址 PC+4。其状态执行路径为：

IF → DAE → WB → IF

在 IF 状态下，CPU 将 jal 指令锁存到 IR 中。由于 jal 是跳转指令，IF 状态下不提前更新 PC，避免错误地进入顺序下一条指令。

在 DAE 状态下，顶层逻辑解析 J-type 偏移量 Offset20 以及目标寄存器字段 rd。ControlUnit 设置 NPCOp 选择 J-type 跳转目标地址，并置 PCWrite=1，使 PC 更新为跳转目标。同时，ControlUnit 设置 WDSel 选择 PCA4，并置 RFWrite=1，使返回地址 PC+4 写回寄存器堆。当前代码中 RegSel 默认选择 rd，因此返回地址写入指令字段指定的目标寄存器。

在 WB 状态下，jal 不再执行寄存器写回，也不再更新 PC，主要用于配合 InsMemRW=1 读取跳转目标地址处的下一条指令。

5.8.8 jalr 指令控制流程

jalr 指令也是无条件跳转并链接指令，但它的跳转目标由寄存器基地址和立即数共同决定。其状态执行路径为：

IF → DAE → WB → IF

在 IF 状态下，CPU 将 jalr 指令锁存到 IR 中。由于 jalr 属于控制转移类指令，IF 状态下不提前更新 PC。

在 DAE 状态下，顶层逻辑解析 rs1、rd 和立即数字段。RF 读出 rs1 对应的基址寄存器值 RD1，EXT 生成扩展后的立即数 Imm32。ControlUnit 设置 ALUSrcA 选择 RD1，ALUSrcB 选择 Imm32，ALUOp 设置为 ADD，使 ALU 计算：

跳转目标 = RD1 + Imm32

该结果以 ALU_result 的形式直接送入 NPC 的 rs_target 端口，用于生成寄存器间接跳转目标。与此同时，ControlUnit 设置 NPCOp 选择 rs_target，置 PCWrite=1，使 PC 更新为 jalr 的目标地址；同时设置 WDSel 选择 PCA4，并置 RFWrite=1，将返回地址 PC+4 写回寄存器

堆。

在 WB 状态下, jalr 不再执行寄存器写回, 也不再更新 PC, 主要用于配合 InsMemRW=1 读取跳转目标地址处的下一条指令。

5.8.9 各类指令控制流程对比表

指令类型	状态路径	DAE 状态主要控制	MEM 状态主要控制	WB 状态主要控制	PC 更新阶段
R-type	IF → DAE → WB	ALUSrcA=RD1, ALUSrcB=RD2, ALUOp 由 Funct3/Funct7 决定	无	RFWrite=1, WDSel=ALU_result_r	IF
I-type	IF → DAE → WB	ALUSrcA=RD1, ALUSrcB=Imm32, ALUOp 由 Funct3 决定	无	RFWrite=1, WDSel=ALU_result_r	IF
lw	IF → DAE → MEM → WB	ALUSrcA=RD1, ALUSrcB=Imm32, ALUOp=ADD	DMCtrl=读	RFWrite=1, WDSel=DR_out	IF
sw	IF → DAE → MEM	ALUSrcA=RD1, ALUSrcB=Imm32, ALUOp=ADD	DMCtrl=写	无	IF
beq	IF → DAE → WB	ALUSrcA=RD1, ALUSrcB=RD2, ALUOp=SUB, 根据 zero=1 决定 NPCOp, PCWrite=1	无	RFWrite=0, InsMemRW=1	DAE
bne	IF → DAE → WB	ALUSrcA=RD1, ALUSrcB=RD2, ALUOp=SUB, 根据 zero=0 决定 NPCOp, PCWrite=1	无	RFWrite=0, InsMemRW=1	DAE
jal	IF → DAE → WB	NPCOp 选择 J-type 跳转目 标, PCWrite=1, RFWrite=1, WDSel=PCA4	无	RFWrite=0, InsMemRW=1	DAE
jalr	IF → DAE → WB	ALUSrcA=RD1, ALUSrcB=Imm32, ALUOp=ADD, NPCOp 选择 rs_target, PCWrite=1, RFWrite=1, WDSel=PCA4	无	RFWrite=0, InsMemRW=1	DAE

表 5-39 流程对比表

控制信号总表

控制信号	控制对象	主要有效阶段	主要作用	设计思路
PCWrite	PC	IF、DAE	控制 PC 更新	普通顺序指令在 IF 阶段提前更新，分支/跳转类指令在 DAE 阶段确定目标后更新
IRWrite	IR	IF	控制指令写入 IR	只在 IF 状态锁存指令，后续状态保持 out_ins 稳定
InsMemRW	IM	IT、DAE、WB	控制指令存储器读取准备	IT 用于初始取指，普通顺序指令在 DAE 预取，分支/跳转类指令在 WB 读取目标地址指令
RFWrite	RF	WB、DAE	控制寄存器写回	R-type、I-type、lw 在 WB 写回，jal/jalr 在 DAE 写回 PCA4
DMCtrl	DM	MEM	控制数据存储器读写	lw/sw 只在 MEM 状态访问 DM
ExtSel	EXT	DAE	控制立即数扩展方式	addi/lw/sw/jalr 采用符号扩展，ori 采用零扩展
ALUOp	ALU	DAE	控制 ALU 运算类型	统一完成算术逻辑运算、地址计算、分支比较和 jalr 目标计算
NPCOp	NPC	IF、DAE	控制下一 PC 来源	普通顺序指令选择 PC+4，分支/跳转类指令在 DAE 选择目标地址
ALUSrcA	MUX_2to1_A	DAE	选择 ALU A 端输入	当前主输入为 RD1，另一输入 5'h0 作为保留
ALUSrcB	MUX_3to1_B	DAE	选择 ALU B 端输入	支持 RD2、Imm32、Offset 三类输入
RegSel	MUX_3to1	RFWrite 有效时	选择写回寄存器地址	默认选择 rd，同时保留 x0、x31 输入
WDSel	MUX_3to1_LMD	WB、DAE	选择写回数据来源	R/I 写回 ALU 结果，lw 写回访存数据，jal/jalr 写回 PC+4

表 5-40 控制信号总表

6. 数据通路与控制信号配合分析

6.1 取指路径数据流与控制信号配合

取指路径主要由 PC、NPC、IM 和 IR 组成，其基本数据流为：

PC → IM → IR

NPC → PC

PC 输出当前指令地址，顶层代码中 IM 的地址输入为 PC[11:2]。由于 RISC-V 指令长度为 32 位，即 4 字节对齐，因此 PC 的低两位不参与字地址访问，实际取指地址使用 PC[11:2]。

IM 根据该地址输出 32 位指令 in_ins，随后送入 IR。

IT 状态用于启动初始取指，此时 InsMemRW=1，使指令存储器输出初始指令。随后状态机进入 IF 状态，ControlUnit 置 IRWrite=1，将 IM 输出的 in_ins 锁存到 IR 中，形成稳定的 out_ins。后续 DAE、MEM 和 WB 等状态都基于 out_ins 完成字段解析和指令执行。

对于 R-type、I-type、lw 和 sw 等普通顺序执行指令，ControlUnit 会在 IF 状态根据 in_ins[6:0] 判断指令类型，并置 PCWrite=1，使 PC 提前更新为 PC+4。对于 beq/bne、jal/jalr 等控制转移指令，IF 状态下不提前更新 PC，而是在 DAE 状态确定分支或跳转目标后，再通过 PCWrite=1 更新 PC。

模块	数据信号	控制信号	作用
PC	输出 PC	PCWrite	普通指令在 IF 状态更新，分支/跳转在 DAE 状态更新
IM	输入 PC[11:2]，输出 in_ins	InsMemRW	在 IT、普通指令 DAE、分支/跳转 WB 等状态配合取指
IR	输入 in_ins，输出 out_ins	IRWrite	只在 IF 状态锁存当前指令
NPC	输入 PC、Offset、Offset20、rs_target 等，输出 NPC	NPCOp	生成顺序地址、分支目标或跳转目标

表 6-1 模块信号说明

6.2 译码路径数据流与控制信号配合

译码路径主要由 IR 输出的 out_ins、字段解析逻辑、RF 寄存器堆和 EXT 立即数扩展模块组成。其主要数据流为：

out_ins → opcode / Funct3 / Funct7 / rs1 / rs2 / rd / Imm12 / Offset20 / Offset

rs1、rs2 → RF → RD1、RD2

Imm12、Offset → EXT → Imm32

当 IR 输出稳定的 out_ins 后，顶层组合逻辑从指令中提取各个字段。其中，opcode、Funct3 和

Funct7 输入 ControlUnit，用于判断当前指令类型和具体操作；rs1、rs2 输入 RF，作为两个源寄存器读地址；rd 输入写回地址选择器，作为普通写回指令的目标寄存器地址；Imm12、Offset20 和 Offset 用于生成立即数或跳转偏移量。

译码相关工作主要在 DAE 状态中与执行逻辑配合完成。字段解析、寄存器读出和立即数准备都属于组合逻辑，不需要额外写使能信号。RF 根据 rs1 和 rs2 输出 RD1、RD2，其中 RD1 和 RD2 会直接作为 ALU 输入选择器的主要数据来源；同时，RD1 和 RD2 也分别送入 U_A、U_B 锁存，其中 RD2_r 主要用于 sw 指令在 MEM 状态向数据存储器写入数据。

数据或模块	来源	去向	相关控制信号	说明
opcode	out_ins[6:0]	ControlUnit	无直接控制	判断当前执行指令类型
Funct3	out_ins[14:12]	ControlUnit	无直接控制	判断 ALU 运算、分支类型或扩展方式
Funct7	out_ins[31:25]	ControlUnit	无直接控制	区分 add/sub、srl/sra
rs1	out_ins[19:15]	RF	无直接控制	源寄存器 1 地址
rs2	out_ins[24:20]	RF	无直接控制	源寄存器 2 地址
rd	out_ins[11:7]	MUX_3to1	RegSel	写回地址候选
Imm32	EXT	MUX_3to1_B	ExtSel	扩展后的立即数

表 6-2 模块信号说明

6.3 执行路径数据流与控制信号配合

执行路径主要由 MUX_2to1_A、MUX_3to1_B、ALU 和 U_ALUOut 组成，同时也包含 U_A、U_B 两个锁存寄存器。其主要数据流为：

RD1 → MUX_2to1_A → A → ALU

RD2 / Imm32 / Offset → MUX_3to1_B → B → ALU

ALU_result → U_ALUOut → ALU_result_r

ALU_result → NPC.rs_target

RD2 → U_B → RD2_r → DM.WD

当前顶层代码中 ALU 的主输入直接来自 RF 输出的 RD1 和 RD2。MUX_2to1_A 的输入为

RD1 和常数 5'h0，由 ALUSrcA 控制；MUX_3to1_B 的输入为 RD2、Imm32 和 Offset，由 ALUSrcB 控制。

对于 R-type 指令，ALUSrcA 选择 RD1，ALUSrcB 选择 RD2，ALUOp 根据 Funct3/Funct7 决定具体运算。对于 I-type 指令，ALUSrcA 选择 RD1，ALUSrcB 选择 Imm32。对于 lw/sw 指令，ALU 用于计算访存地址，即 $RD1 + Imm32$ 。对于 beq/bne 指令，ALU 执行 $RD1 - RD2$ ，并通过 zero 判断两个寄存器是否相等。对于 jalr 指令，ALU 计算 $RD1 + Imm32$ ，其组合输出 ALU_result 直接送入 NPC 的 rs_target 端口，用于生成寄存器间接跳转目标。

模块	输入来源	输出	控制信号	功能
U_A	RD1	RD1_r	clk/rst	锁存源操作数 1，当前主要作为保留路径
U_B	RD2	RD2_r	clk/rst	锁存源操作数 2，主要用于 sw 写入 DM
MUX_2to1_A	RD1、5'h0	A	ALUSrcA	选择 ALU A 端输入
MUX_3to1_B	RD2、Imm32、Offset	B	ALUSrcB	选择 ALU B 端输入
ALU	A、B	ALU_result、zero	ALUOp	运算、比较、地址计算、jalr 目标计算
U_ALUOut	ALU_result	ALU_result_r	clk/rst	锁存 ALU 结果，供 MEM/WB 使用

表 6-3 模块信号说明

6.4 访存路径数据流与控制信号配合

访存路径主要由 U_ALUOut 和 DM 数据存储器组成，只对 lw 和 sw 指令有实际作用。其数据流为：

$ALU_result_r[11:2] \rightarrow DM.Addr$

$RD2_r \rightarrow DM.WD$

$DM.RD \rightarrow DR_out$

lw 和 sw 指令都需要先在 DAE 状态计算有效地址。该地址由 ALU 完成计算，并由 U_ALUOut 锁存为 ALU_result_r。进入 MEM 状态后，DM 使用 ALU_result_r[11:2] 作为访问地址。这里取 [11:2] 是因为数据按字访问，低两位表示字节偏移，对 32 位字访问来说可以忽略。

对于 lw 指令，MEM 状态下 DMCtrl 设置为读控制，DM 根据地址输出数据 RD。当前顶层代码中，U_DR 被注释掉，采用 assign DR_out = RD; 的方式，使 DR_out 直接等于数据存储器读出数据。随后 DR_out 会在 WB 状态通过写回数据选择器写入寄存器堆。

对于 sw 指令，MEM 状态下 DMCtrl 设置为写控制，DM 将 RD2_r 写入 ALU_result_r[11:2] 对应地址。由于 sw 指令不需要写回寄存器堆，因此没有 WB 状态。需要注意的是，当前版本中 sw 的 PC 已经在 IF 状态提前更新，因此 MEM 状态不再需要置 PCWrite=1。

指令	地址来源	写数据来源	读数据去向	控制信号
lw	ALU_result_r[11:2]	无	RD → DR_out → MUX_3to1_LMD	DMCtrl 读控制
sw	ALU_result_r[11:2]	RD2_r	无	DMCtrl 写控制

表 6-4 模块信号说明

6.5 写回路径数据流与控制信号配合

写回路径主要由 MUX_3to1、MUX_3to1_LMD 和 RF 组成。其基本数据流为：

rd / 5'd0 / 5'd31 → MUX_3to1 → WR

ALU_result_r / DR_out / PCA4 → MUX_3to1_LMD → WD

WR、WD → RF

写回地址选择器 MUX_3to1 由 RegSel 控制。普通 R-type、I-type 和 lw 指令通常选择 rd 作为写回寄存器地址；5'd0 可作为无效写回或保留路径；5'd31 可作为链接寄存器写回的保留输入。需要注意的是，当前 ControlUnit 中 RegSel 默认选择 rd，因此如果没有额外设置 RegSel 选择 x31，jal/jalr 的返回地址会写入指令字段 rd，而不是固定写入 x31。

写回数据选择器 MUX_3to1_LMD 由 WDSel 控制。R-type 和 I-type 指令选择 ALU_result_r，lw 指令选择 DR_out，jal/jalr 指令选择 PCA4。这样所有写回数据都通过同一个 WD 端口进入 RF，RF 不需要关心写回数据来自 ALU、存储器还是 PC+4。

写回真正发生的条件是 RFWrite=1。如果 RFWrite=0，即使 WR 和 WD 有对应值，也不会写入寄存器堆。当前版本中，R-type、I-type 和 lw 在 WB 状态写回；jal/jalr 的 PCA4 写回发生在 DAE 状态，而不是 WB 状态。

指令类型	RegSel	WDSel	RFWrite 有效阶段	写回内容
R-type	rd	ALU_result_r	WB	ALU 运算结果
I-type	rd	ALU_result_r	WB	ALU 运算结果
lw	rd	DR_out	WB	数据存储器读出数据
sw	无实际写回	无实际写回	无	不写回
beq/bne	无实际写回	无实际写回	无	不写回
jal/jalr	默认 rd, 可扩展为 x31	PCA4	DAE	返回地址 PC+4

表 6-5 模块信号说明

6.6 典型指令执行过程分析

不同指令在数据通路中的主要差异体现在 ALU 输入选择、访存路径、写回数据来源和 PC 更新阶段上。

指令	状态路径	ALU 输入	ALU 功能	访存	写回	PC 更新
add	IF → DAE → WB	RD1、RD2	RD1 + RD2	无	ALU_result_r → rd	IF
addi	IF → DAE → WB	RD1、Imm32	RD1 + Imm32	无	ALU_result_r → rd	IF
lw	IF → DAE → MEM → WB	RD1、Imm32	计算访存地址	DM 读	DR_out → rd	IF
sw	IF → DAE → MEM	RD1、Imm32	计算访存地址	RD2_r → DM	无	IF
beq	IF → DAE → WB	RD1、RD2	RD1 - RD2, zero=1 时跳转	无	无	DAE
bne	IF → DAE → WB	RD1、RD2	RD1 - RD2, zero=0 时跳转	无	无	DAE
jal	IF → DAE → WB	不依赖 ALU 主运算	NPC 根据 Offset20 生成跳转目标	无	PCA4 → rd, 在 DAE 写回	DAE
jalr	IF → DAE → WB	RD1、Imm32	RD1 + Imm32 生成 rs_target	无	PCA4 → rd, 在 DAE 写回	DAE

表 6-6 数据通路说明

通过以上分析可以看出,当前版本的数据通路以 RD1/RD2、Imm32、ALU_result_r、DR_out 和 PCA4 为核心数据来源,通过 ALUSrcA、ALUSrcB、ALUOp、DMCtrl、RegSel、WDSel 和

RFWrite 等控制信号完成不同类型指令的数据选择与状态推进。DAE 合并了译码与执行功能，普通顺序指令在 IF 状态提前更新 PC，而分支和跳转指令在 DAE 状态完成 PC 更新。

7 系统设计总结

7.1 设计完成情况

本设计完成了一个基于 RISC-V 指令集的多周期 CPU。系统采用 Verilog HDL 描述，顶层模块结构清晰，子模块划分明确，能够完成取指、指令锁存、译码执行、访存和写回等基本 CPU 操作。当前设计中，控制单元采用 IT、IF、DAE、MEM、WB 五个状态，其中 DAE 状态将传统意义上的译码和执行功能合并到同一阶段完成，使控制流程更加集中。

7.2 系统结构特点

本设计最大的特点是采用多周期结构，不同指令根据功能需求经过不同状态路径。该结构能够复用 ALU、寄存器堆和存储器等硬件资源，避免单周期 CPU 中所有指令都受最长路径限制的问题。同时，通过 DAE 状态合并译码与执行过程，使普通运算、地址计算、分支判断和跳转控制都能在统一的执行状态中完成，进一步简化了状态机控制逻辑。

7.3 控制信号设计特点

ControlUnit 采用有限状态机进行控制，所有控制信号均由当前状态和指令字段共同决定。设计中对 PCWrite、IRWrite、InsMemRW、RFWrite、DMCtrl、ExtSel、ALUOp、NPCOp、ALUSrcA、ALUSrcB、RegSel 和 WDSel 等重点控制信号进行了明确划分，每个信号都有对应的作用阶段和设计目的。

PCWrite 的控制方式较为关键：R-type、I-type、lw 和 sw 等普通顺序指令在 IF 状态提前更新 PC，而 beq/bne、jal 和 jalr 等控制转移指令则在 DAE 状态完成目标地址判断或计算后更新 PC。RFWrite 也根据指令类型分阶段生效，其中 R-type、I-type 和 lw 在 WB 状态写回，jal/jalr 则在 DAE 状态写回 PCA4。

7.4 数据通路设计特点

数据通路采用模块化设计，PC、NPC、IM、IR、RF、EXT、ALU、DM 和 MUX 等模块通过

清晰的信号连接构成完整 CPU。当前设计中，ALU 的主输入直接来自寄存器堆输出的 RD1 和 RD2，其中 RD1 送入 MUX_2to1_A，RD2 送入 MUX_3to1_B；U_A 和 U_B 仍然保留锁存路径，其中 RD1_r 主要作为辅助锁存或保留路径，RD2_r 主要用于 sw 指令在 MEM 状态向数据存储器写入数据。U_ALUOut 则用于锁存 ALU_result，生成 ALU_result_r，供后续访存地址访问或普通运算结果写回使用。

7.5 总结

总体来看，本设计实现了一个结构清晰、控制逻辑明确、数据通路完整的 RISC-V 多周期 CPU。系统以 ControlUnit 为控制核心，通过 IT、IF、DAE、MEM 和 WB 五个状态管理不同类型指令的执行过程，并通过数据通路中的寄存器、多路选择器、ALU、存储器和写回选择器完成操作数传递、运算处理、访存访问、寄存器写回和 PC 更新。